PHP Gallery

Complete Product Documentation

Installation, administration, architecture, security, maintenance, and development reference

Application version:	0.90
Manual edition:	16 August 2026
Document format:	A4, indexed, linked PDF
Project author:	Rudolf Klusal
License:	MIT License

A practical guide to creating, organizing, presenting, protecting, sharing, and preserving a photographic collection with PHP Gallery version 0.90.

Contents

1 Purpose and reading guide

PHP Gallery is a website for presenting photographs as organized collections. You choose where it runs, keep control of the original files, and decide who can see each gallery. It can serve a public portfolio, a family archive, a private client delivery area, an event collection, a travel diary, or a large personal photographic library.

This manual is written for the person who owns or manages that collection. It explains what each feature is useful for, where to find it, how choices affect visitors, and what to check afterward. You can read it from beginning to end or use the linked contents and index to answer a specific question.

Start with Section 5 when the site has just been installed. Section 6 helps you plan a collection before adding hundreds of photographs. Sections 7 and 9 explain ordinary gallery work. Section 10 provides complete examples for common audiences. Technical background remains near the end for readers who want to understand hosting, backups, or how the application works behind the screen.

1.1 What is new in Version 0.90

Version 0.90 adds Smart Galleries, browser-local ZIP photo import, and optional multilingual gallery and photo metadata. A Smart Gallery is a securely validated saved rule tree whose current results are drawn from accessible physical galleries without copying or moving files. Administrators can combine nested AND, OR, and NOT groups, preview matches, publish stable URLs, place one Smart Gallery at the root or beneath several physical galleries, and use private editorial ratings independently from public voting.

Browser-assisted upload can now inspect ordinary stored or Deflate ZIP exports locally, add supported images to the existing preparation queue, and skip invalid or unsupported entries without sending the selected archive to PHP. Gallery and photo editors can also classify source text and save reviewed title/description variants for English, Czech, German, and Swedish. The viewer's browser-local language preference selects matching content when available; access rules, paths, slugs, filenames, and source fields remain unchanged.

Four ordered migrations add Smart Gallery definitions, editorial ratings, flexible placements, and multilingual content storage. Existing installations use the normal updater and migration runner; neither Smart Gallery membership nor translated presentation requires copying images or rebuilding metadata. Version 0.90 retains hourly stable-release discovery, resumable updater jobs, configurable public language-selector designs, query-string routing, no-JavaScript fallbacks, and shared-hosting compatibility.

1.2 How to use this manual

Each feature is presented in four practical parts whenever appropriate:

1. its purpose and the problem it solves;
2. a common situation in which it is useful;
3. the choices available to the gallery owner;
4. the result a visitor should experience.

Names shown in bold, such as **Theme** or **System Health**, refer to areas or controls in the Admin interface. Text in a monospaced style usually represents a file name, setting name, or command and appears mainly in the optional technical sections.

1.3 Further project references

This manual gathers the information needed by gallery owners into one reading experience. The project also includes concise documents for developers and server administrators. They are listed in the References section and cited where they provide useful supporting detail [1, 2, 3, 4, 6, 7].[1]

1.4 Terminology

Gallery A named collection of photographs. A gallery can contain photographs and smaller galleries.

Photograph or image One item in a gallery, together with its title, description, tags, visibility, and available camera information.

Thumbnail A smaller copy created for quick display in gallery grids, search results, covers, and previews.

Visitor A person viewing the public side of the site.

Administrator A trusted person who signs in to create and manage content and settings.

Selected gallery The gallery currently open on the screen.

Gallery branch A gallery together with every smaller gallery contained beneath it.

[1] The additional project documents are useful when modifying application code, examining database structure, or preparing a software release. Ordinary gallery creation and daily administration can be learned entirely from this manual.

2 Product overview

PHP Gallery turns a folder collection into a browsable photographic website. Visitors see covers, titles, descriptions, photo grids, full-screen viewing, maps, tags, search, and downloads according to the choices made by the owner. Administrators receive private tools for adding photographs, arranging collections, controlling privacy, improving descriptions, and keeping the installation healthy.

The application is designed to run on ordinary web hosting. This matters to an owner because the gallery can live on a familiar hosting account and remain under the owner's control. The original photographs stay in recognizable folders, and the database remembers the information that makes those files pleasant to browse.[2]

2.1 Core capabilities

- Create galleries inside other galleries, for example *2026 / Italy / Rome* or *Clients / Northwind / Final selection*.
- Give every gallery a title, description, date range, cover, public address, privacy choice, and individual appearance.
- Upload photographs through the browser or discover files copied into gallery folders.
- Add titles, stories, tags, visibility, ordering, dates, and location information to photographs.
- Let visitors search, browse tags, view photographs full screen, follow maps, vote, and download permitted collections.
- Protect family, client, or sensitive collections with access controls and share methods.
- Find likely duplicate photographs and review them with direct links to both locations.
- Keep the installation current through guided updates, health checks, backups, and maintenance tools.

2.2 Filesystem and database partnership

Think of the installation as a photographic cabinet with a catalog. The gallery folders are the cabinet: they hold the actual image files in an understandable arrangement. The database is the catalog: it remembers titles, descriptions, tags, display order, privacy, votes, dates, and other details.

Both parts are valuable. If the files were present without the catalog, the site would lose much of its organization and presentation. If the catalog were present without the files, it would describe photographs that were unavailable. This is why a complete backup always includes the gallery folders and the database together.[3]

[2] The project overview provides a concise feature and installation summary [1].
[3] Section 11 explains this partnership in everyday language and provides a complete backup model.

3 Requirements and environment

This section is mainly for the person choosing a hosting plan or helping with installation. A gallery owner can send this page to the hosting provider and ask whether the account meets the listed requirements.

3.1 Minimum runtime

Component	Requirement
PHP	Version 8.0 or newer.
Database	MySQL 5.7 or newer, or MariaDB 10.2 or newer.
PDO	PDO MySQL extension for application queries and migrations.
Image processing	GD for common thumbnail generation. Additional local image tooling can support specialized formats.
Archives	ZipArchive for package, download, and transfer workflows that produce or consume ZIP data.
Filesystem	Writable application data, gallery, cache, and generated-media locations.
Web server	Apache-style rewriting is recommended for clean URLs; query-string routing supports compatible environments without rewrites.

Outbound HTTPS access supports application updates and remote release metadata. Local operation and manually deployed releases can function with a more restricted outbound policy.[4]

3.2 Permissions

The web process needs read access to runtime code and write access to designated mutable locations. Keep runtime code read-only where hosting permits it, except during a controlled updater operation. Gallery source files deserve backup coverage equal to the database because the complete application state spans both domains.

Important. Create backups of the database, gallery files, and installation configuration together. A database-only backup preserves metadata while omitting the authoritative media tree.

[4] Network policy should follow the installation owner's update and privacy requirements. Update code validates packages and preserves recovery data according to the local updater implementation.

4 Installation and initial setup

4.1 Browser installation

4.1.1 Installation sequence

The browser installer gathers database settings, initializes the schema through migrations, creates the first administrator, prepares writable locations, and writes local configuration. A fresh request redirects to installation when required configuration is absent.[5]

Typical steps are:

1. Create an empty MySQL or MariaDB database using the hosting control panel.
2. Upload the application package to the intended web directory.
3. Open the site and follow the installer.
4. Enter database connection values and the initial administrator credentials.
5. Confirm writable directories and extension checks.
6. Complete installation and enter the Admin area.

4.2 Command-line setup

4.2.1 Local commands

The repository provides plain PHP scripts for migration and administrator creation:

```
php scripts/migrate.php
php scripts/create_admin.php <username> <password>
```

Command-line setup follows the same persistent schema model as browser setup. Review `config.example.php`, create the local configuration, provision the database, run migrations, and create the first administrator.

4.3 First administrative review

After installation, review:

- Site name, language, theme, page width, and public rendering preferences.
- Gallery root permissions and discovery results.
- Thumbnail formats, dimensions, quality, and maintenance state.
- Access, password-reset, email, telemetry, update-channel, and backup preferences.
- Rewrite compatibility and canonical public URLs.
- System Health diagnostics and pending migrations.

[5] Installation state and generated configuration should receive the same protection as credentials. Source archives should continue excluding the local `config.php`.

5 Your first hour with PHP Gallery

This section follows the experience of a new gallery owner after installation. The goal is a small, attractive, working gallery that can be viewed from another browser. You can refine colors, descriptions, privacy, and organization after this first success.

5.1 Open the Admin area

Choose **Admin login** from the public header and enter the account created during installation. The dashboard is the starting point for everyday ownership. It summarizes galleries, photos, system state, and available maintenance actions.[6]

Spend a few minutes identifying these areas:

- **Galleries**, where collections are created, discovered, edited, and arranged;
- **Theme**, where the public site's appearance and presentation defaults are selected;
- **Tags**, where reusable subjects and categories are maintained;
- **System Health**, where migrations, thumbnails, storage, and consistency are reviewed;
- **Account**, where credentials, recovery email, and mail delivery are managed.

5.2 Create a first gallery

Open the gallery administration area and choose the action for creating a gallery. Give it a short, recognizable title such as *Summer Walk*. A simple title works well because the application can propose a public path and folder name from it. Add a one-sentence description that tells a visitor what the collection contains.

Select public visibility for this first exercise. Leave advanced branding and inherited settings at their defaults. Save the gallery, open it, and add three to ten photographs. A compact first collection makes it easy to see ordering, thumbnail generation, titles, and the lightbox without waiting for a large upload.

5.3 Preview as a visitor

Use the public-gallery link or visitor-preview control. For a realistic check, open the public address in a private browser window where the Admin session is absent. Confirm that the title, description, photographs, and navigation appear as intended. Open a photograph, move forward and backward in the viewer, and return to the gallery.

Administrator views can contain edit controls and additional assets. Anonymous preview gives the clearest picture of the experience received by family, clients, or the public.[7]

[6] The exact dashboard contents depend on enabled features, completed migrations, and development settings. A healthy installation can still display informational notices that invite an administrator to complete optional work.

[7] Protected galleries and age-restricted content deserve a separate anonymous test because an active administrator session already has broader access.

5.4 Complete the first-hour checklist

1. Set the public site name and language.
2. Create one gallery with a title and description.
3. Upload a small group of photographs.
4. Add a meaningful title to at least one photograph.
5. Reorder two photographs and confirm the new order publicly.
6. Open the gallery as an anonymous visitor.
7. Check the gallery on a phone-sized screen.
8. Confirm that System Health reports no urgent migration or permission problem.
9. Create a coordinated backup after the initial setup is complete.

6 Planning a gallery collection

A thoughtful structure helps visitors understand a collection and helps the owner maintain it over time. Start with the way people will look for photographs. Folder names and database details can follow that human organization.

6.1 Choose a structure people recognize

Common structures include:

By year and event A top-level year contains weddings, trips, celebrations, and projects. This suits family and historical archives.

By place Countries contain regions and cities. This suits travel, aviation, landscape, and documentary photography.

By client or project Each client contains assignments and delivery galleries. This suits professional work with separate access rules.

By subject Wildlife, architecture, portraits, aircraft, and similar subjects form enduring categories. Tags can provide a second way to browse across them.

By workflow Incoming, selected, delivered, and archive galleries represent stages. Visibility settings keep working collections private while finished selections become public.

Use galleries for strong navigational boundaries and tags for relationships that cross those boundaries. A photograph from Prague can live in *Travel / Czech Republic / Prague* and carry tags such as *architecture*, *night*, and *tram*.[8]

6.2 Decide how deep the hierarchy should be

Nested galleries support deep collections, yet most visitors browse more comfortably when each level has a clear purpose. Two to four levels serve many sites well. A deeper archive remains practical when breadcrumbs, meaningful titles, and consistent dates guide the reader.

Create a child gallery when it deserves its own title, description, cover, access policy, date range, or public address. Keep photographs together when a new level would add only one extra click.

6.3 Prepare titles and descriptions

A gallery title should make sense outside the Admin interface. A description can answer three questions:

- What does this collection show?
- Where and when was it created?
- What context makes the photographs meaningful?

Descriptions can be brief for casual albums and extensive for documentary work. Use the first sentence as the essential summary because visitors often scan before reading the full text.

[8] A photograph has one gallery location in the normal content model, while reusable tags provide several semantic connections. Copy and move tools support deliberate changes to physical organization.

6.4 Plan privacy before uploading

Choose whether a collection is public, unpublished for administrative preparation, or protected for a known audience. Parent-gallery policy can influence descendants, so place collections with similar audiences together where practical.

Examples include:

- a public portfolio with carefully selected photographs;
- a password-protected family branch;
- an unpublished client proofing gallery during preparation;
- a public event gallery with selected individual photographs marked private;
- an age-gated branch for content that needs visitor confirmation.

7 Creating and editing galleries

7.1 Create a gallery from Admin

Choose a parent gallery when the new collection belongs inside an existing subject, year, place, or client. Leave the parent empty for a top-level gallery. Enter a title, review the proposed folder or public path, and add the description, dates, and initial visibility.

After saving, the gallery becomes a real collection in the gallery tree. Its folder can receive photographs through the browser, filesystem transfer, discovery, or another supported upload workflow. The database stores its descriptive and presentation settings.

7.1.1 A practical creation sequence

1. Choose the intended parent.
2. Enter a visitor-friendly title.
3. Confirm the folder and public-path proposal.
4. Add a description and date range where useful.
5. Select visibility and access.
6. Save the gallery.
7. Add photographs or create child galleries.
8. Select a cover after suitable photographs exist.
9. Preview the result anonymously.

7.2 Edit identity and context

The editor lets you improve a gallery over time. Titles and descriptions can change as a collection develops. Date fields can represent a single day, a trip, a season, or a longer historical period. EXIF date suggestions can examine photographs and descendants, then offer an editable range for approval.

Use date suggestions as a helpful starting point. Scanned photographs, screenshots, edited exports, and cameras with an incorrect clock can produce dates that deserve human review.[9]

7.3 Choose a cover and visual identity

A cover acts as the gallery's visual invitation. Choose an image that remains recognizable as a thumbnail and represents the collection fairly. Faces, strong silhouettes, clear subjects, and uncluttered compositions often work well at small sizes.

Gallery branding can provide a logo, banner, background, separator, or presentation override according to available controls. Begin with inherited theme styling, then add gallery-specific assets where a collection has a genuine identity, such as a wedding, exhibition, client, or long-running project.

[9] Applying an EXIF-derived suggestion updates the proposed gallery fields through the existing editor workflow. The administrator remains responsible for the final dates shown to visitors.

7.4 Move and reorder galleries

Moving a gallery changes its place in the hierarchy and can affect inherited access or presentation. Review the destination before confirming the move. Reordering changes the sequence shown among siblings and supports a curated reading order.

A useful order can be chronological, geographical, narrative, alphabetical, or manually featured. Keep the principle consistent within one parent so visitors can predict what comes next.

7.5 Delete a gallery thoughtfully

Gallery deletion can affect source files, descendants, metadata, thumbnails, links, and visitor bookmarks. Review the exact selected gallery and its children, create a backup, and use the normal Admin deletion workflow. A temporary visibility change can provide a safer preparation period when a gallery may still be needed.

8 Smart Galleries

A Smart Gallery is a saved query, not a folder. Open **Admin > Galleries > Smart Galleries**, add conditions, and combine them with nested **AND**, **OR**, and **NOT** groups. Fields cover source galleries and descendants, tags, capture dates, EXIF/GPS, text, AI metadata, duplicate state, file characteristics, and private editorial ratings.

Use **Preview count**, save, and optionally publish. Images and files remain in their physical galleries, so metadata or rating changes alter membership immediately. The editorial rating is private and independent from visitor voting. Administrators may also use **Save search as Smart Gallery** from public search.

Choose **Unlisted** for URL-only access, **Root gallery** to show the Smart Gallery on the homepage, or **Subgallery placement** to keep it off the homepage while allowing physical galleries to list it. Each physical gallery editor contains a Smart Gallery checklist. The same virtual gallery may be selected beneath any number of physical galleries, and removing one placement does not disturb the others. The Smart Gallery editor's **Used in physical galleries** section lists every attachment, links to each physical gallery editor, and provides an independent **Hide from here** action. It must still be enabled and published before visitors see its card.

Public results and counts always intersect rules with access to the physical source gallery. Query-string routes work without rewriting and clean `/smart/<slug>` URLs are optional. Rules are readable versioned JSON, never user SQL: fields/operators are allowlisted, values are prepared parameters, depth is limited to five, and no more than 50 conditions are accepted. Deleted references match nothing and malformed or unknown rule versions fail safely. The normal migration workflow adds storage and ratings; no metadata rebuild is required.

9 Adding, describing, and organizing photographs

9.1 Choose an upload method

Browser upload suits everyday additions. Select a gallery, choose files, review the batch, and follow progress while the server validates media and prepares records and thumbnails. Filesystem discovery suits large existing folder trees or transfers made through FTP and hosting tools. Automation and WebDAV suit repeated or mobile workflows after their scoped credentials are configured.

For a large event, upload in meaningful groups. Smaller groups make errors easier to identify and allow titles, ordering, and visibility to be reviewed incrementally.

9.2 Write useful photo titles

A title identifies the photograph quickly. A description or caption can explain people, place, activity, technical context, or a story outside the frame. Useful examples include:

- *Charles Bridge before sunrise*;
- *Arrival of the first train at the restored station*;
- *Family group in the garden, June 1998*;
- *Final approach during the Saturday display*.

Meaningful text improves search, accessibility, later identification, and the experience of readers who want more than a visual grid. File names can remain operational details when public display settings favor curated titles.

9.3 Use tags as a second organization layer

Create a small vocabulary before adding many near-duplicate tags. Prefer one stable term, such as *aircraft*, over several accidental variants. Tags can represent subjects, people, places, techniques, equipment, seasons, or editorial status.

For a travel archive, galleries can represent trips and tags can represent recurring themes. For a family archive, galleries can represent years and events while tags identify people. For a professional portfolio, galleries can represent clients and tags can identify deliverable type, location, and specialty.

9.4 Arrange the viewing order

Ordering turns a collection into a sequence. Start with an inviting image, establish place or context, vary wide and close views, keep closely related details together, and finish with a natural conclusion. Chronological order works well for events, while visual rhythm can suit portfolios and exhibitions.

The public reorder controls help administrators adjust visible cards. Pagination can divide a large collection, so review transitions around page boundaries as well as the first page.

9.5 Review EXIF and location information

EXIF can enrich a photograph with capture date, camera, lens, exposure, and GPS coordinates. This information supports maps, date suggestions, duplicate evidence, and technical context. Review GPS visibility before publishing photographs made at homes, schools, private workplaces, or sensitive natural locations.

10 Publishing, sharing, and audience scenarios

10.1 Public portfolio

A public portfolio benefits from a small number of strong top-level galleries, consistent covers, concise descriptions, and careful ordering. Use tags for specialties that cross projects. Keep source archives in private or unpublished branches and copy selected work into public presentation galleries where the workflow calls for separate curation.

Test the portfolio on a phone, a high-density display, and a slower connection. Responsive thumbnail selection provides the default balance. Progressive sharpening can make small thumbnails populate first for installations that prioritize early visual response.

10.2 Private family archive

Organize by year, family branch, or event. Use password-protected parent galleries to establish an understandable private area, then place related descendants within it. Add names and dates while memories are available. Scanned photographs benefit especially from human descriptions because embedded camera metadata may be absent.

Share the protected address and password through an appropriate private channel. Explain that recipients can navigate child galleries without receiving an administrator account. Periodically review whether old links and passwords still serve the intended audience.

10.3 Client delivery

Create one protected gallery per client or assignment. Upload proofs, organize them into useful groups, and use descriptions to explain selection or download expectations. Voting can support lightweight preference gathering when enabled for that gallery. A final delivery gallery can contain approved files in the intended order.

Share links offer another controlled entry method where configured. Test the exact client journey in a private browser window before sending the address.

10.4 Event and community gallery

An event gallery can begin unpublished while photographs are arriving. Use child galleries for days, stages, teams, locations, or activities. Publish the parent when the first useful selection is ready, then add later groups without changing the familiar public address.

Public voting can highlight audience favorites. Tags connect recurring participants or subjects. Downloads provide a convenient collection transfer when the event policy allows it.

10.5 Travel and location archive

Country, region, city, and trip galleries provide natural navigation. Date ranges establish chronology, while GPS maps create a geographical reading path. Review coordinates for accommodation and private locations. A descriptive introduction can explain the route, purpose, and historical context of a trip.

10.6 Historical and institutional archive

Historical collections benefit from careful source notes, uncertain-date wording, names, locations, and provenance. Use descriptions to distinguish confirmed facts from family or institutional recollection. Tags can connect people, buildings, organizations, and periods across many galleries.

Create backups before large metadata projects and export information through available tools when planning external preservation. The gallery can provide an accessible presentation layer while original preservation practices continue alongside it.

11 Understanding your gallery data

Gallery owners interact with two cooperating stores: ordinary files and a database. Understanding their roles makes backup, migration, and troubleshooting easier.

11.1 What lives in gallery folders

The gallery folders contain the photographs and folder hierarchy. This makes the core collection recognizable through FTP, a hosting file manager, a backup tool, or a local copy. Moving or renaming files outside the application can require discovery or synchronization so the database learns the new state.

11.2 What lives in the database

The database remembers titles, descriptions, dates, visibility, passwords and access configuration, public paths, ordering, tags, votes, image dimensions, EXIF indexes, checksums, thumbnail records, administrator accounts, settings, audit information, and workflow state.[10]

The database makes search and administration fast. It also allows a file to have a friendly title, rich description, tags, a chosen order, and access rules without changing the original image bytes.

11.3 How users benefit from the database

Everyday benefits include:

- searching words across gallery and photo descriptions;
- opening a gallery through a stable public path;
- displaying photographs in a curated order;
- remembering tags and visitor votes;
- protecting a gallery with inherited access rules;
- finding exact duplicate content through stored checksums;
- preparing suitable thumbnails without re-reading every source file on every page;
- recording migrations and maintenance decisions safely.

11.4 Backup as one complete set

A complete backup includes the gallery tree, database export, local configuration, and installation-owned custom assets. Give these parts the same backup date so they describe one coherent state. Store a copy away from the web server and test restoration in a separate location.

[10] Password values use hashes or protected configuration appropriate to their purpose. A database export still contains sensitive operational information and deserves secure storage.

12 Everyday ownership and care

12.1 After every important upload

Review the destination gallery, photo count, ordering, titles, visibility, and thumbnail appearance. Open several photographs in the lightbox and check the gallery anonymously. For a protected collection, repeat the password or share-link journey from a fresh private session.

12.2 Monthly review

A practical monthly routine includes:

1. Review System Health and pending migrations.
2. Check recent Admin audit events.
3. Inspect available application updates and their release information.
4. Confirm scheduled or manual backups completed.
5. Open representative public and protected galleries.
6. Review thumbnail maintenance when source files or rendering settings changed substantially.
7. Remove or revoke credentials and share methods that have completed their purpose.

12.3 Before a major public launch

Check titles, descriptions, cover images, spelling, mobile layout, privacy, GPS exposure, downloads, maps, voting, search results, and contact expectations. Test through a slow network profile if the audience may use mobile connections. Create a backup immediately before the launch state.

12.4 When another person helps administer

Give each administrator an individual account where the account model supports the intended workflow. Explain gallery scope, privacy expectations, naming conventions, tag vocabulary, backup responsibilities, and deletion procedures. Individual accounts improve accountability in audit records and keep personal duplicate-review ledgers separate.

13 Public visitor experience

13.1 What a visitor sees

A visitor opens the site at the home page or follows a direct link to a gallery or photograph. The page presents the site's name, navigation, gallery title, description, breadcrumbs, tags, child galleries, and photographs that the visitor is allowed to see. Protected content asks for the appropriate access step before revealing private material.

Each gallery has a public address that can be bookmarked or shared. A direct photo address opens the gallery context and leads the visitor to that photograph. Meaningful titles and stable public paths help links remain understandable in messages, bookmarks, and search results.

13.1.1 A normal visit

A typical visitor journey looks like this:

1. Open the home page and choose an interesting gallery cover.
2. Read the gallery introduction and follow child galleries or tags.
3. Scroll through smaller preview images.
4. Open one photograph in the large viewer.
5. Move through nearby photographs with buttons, keys, a strip, or the selected viewing mode.
6. Open a map, vote, search, or download when the gallery offers those actions.
7. Use breadcrumbs to return to a broader collection.

13.2 Gallery hierarchy and pagination

13.2.1 Direct content and stable ordering

A gallery can show smaller galleries first and photographs afterward. For example, *Italy 2026* can contain *Rome*, *Florence*, and *Venice*, while also showing a few overview photographs directly on the Italy page.

Pagination divides a long collection into manageable pages. This helps the first page appear promptly and keeps scrolling comfortable. A small exhibition may look best on one page. A wedding with 1,500 photographs will usually benefit from pages or child galleries. The large photo viewer can continue through the wider gallery sequence while loading additional information as needed.[11]

13.3 Search, tags, and navigation

Search can look across the whole public site or stay within the current gallery and its children. A visitor looking at *Family archive* can search only that branch for a person's name, while a visitor on the home page can search across every public collection.

[11] Pagination changes how many cards appear at once. It leaves the underlying photographs and their order intact.

Tags provide another route through the archive. A *night photography* tag can connect Prague, London, and Tokyo even when those photographs live in separate travel galleries. Breadcrumbs show the current place in the hierarchy, and favorite shortcuts place important galleries directly in the site header.

14 Media and gallery administration

14.1 Admin workspace

After signing in, an administrator sees private controls for the dashboard, galleries, photographs, theme, tags, maintenance, updates, logs, and account settings. Many editing actions open in a panel on the right so the gallery remains visible in the background. This makes it easier to edit one item, save it, and continue working through the same collection.[12]

14.2 Central Settings hub

The Admin navigation includes a central **Settings** page for discovering important site-wide configuration without replacing the existing specialist screens. The page is divided into a small set of peer sections: General, Public appearance, Content, Media and browsing, Uploads and automation, Privacy and diagnostics, and Advanced. Each section has a matching visible heading and a stable URL, so bookmarks, browser history, refresh, keyboard navigation, and the no-JavaScript fallback all lead back to the same category.

Immediately below the Settings title, a Spotlight-style search field searches the complete central registry as text is entered. It matches translated setting names, descriptions, categories, and stable technical names without sending a request to the server. A short query such as `thum` can therefore reveal thumbnail rendering, thumbnail checks, and related media tools even when they live in different parts of the interface. Choosing a result opens its owning section and focuses the exact setting or status card. The result list supports the arrow keys, Enter, Escape, and a clear button; the ordinary section navigation remains the no-JavaScript fallback.

The search registry includes each global control from the specialist Admin screens, not merely the small centrally editable subset. Theme colors, dimensions, typography, map pins, gallery cards, grids, shortcuts, branding media, language tools and custom CSS; upload limits; telemetry collection and retention; thumbnail and scheduled-maintenance tools; navigation data; logs, updates, integrity, diagnostics, reports, features and database operations; and Account mail, linked-login and AI-assistance controls all have individual discovery entries. These entries route to the screen that owns validation and saving, and do not duplicate hundreds of controls in the normal Settings section panels. Gallery-specific and photograph-specific values remain in their contextual editors, and secret values themselves are never indexed.

The hub deliberately does not behave like one enormous form. Safe settings with an existing canonical service setter can be edited in their own small section form. Current central editing covers site name, public language, URL rewrite, public search when available, the public thumbnail renderer, the global EXIF/GPS display default when its existing schema is ready, and development diagnostics. Saving one group does not submit unrelated settings.

More contextual configuration remains on the page that already owns it. Theme layout, Gallery tags presentation, uploads, telemetry, Account passwords, Google and OpenAI credentials, raw CSS, branding uploads, language packs, API keys, database repair and destructive maintenance stay specialized. The central hub shows safe status and current-value summaries and provides explicit links such as **Open specialized page**. Specialist pages also provide **Open centralized settings** links back to the relevant section.

Tag-page presentation retains inheritance instead of silently creating new values. Dedicated tag columns and rows inherit the global pagination dimensions when absent, and the tag-page card

[12] A full Admin page remains available for workflows that need more space or when browser enhancement is unavailable.

layout inherits the global Theme card layout. Hero-tag display controls are separate. Gallery-specific card layout, lightbox and EXIF/GPS overrides remain attached to each gallery and are not rewritten by the global Settings page.

Secrets are never displayed as raw summaries. SMTP passwords, maintenance tokens, OpenAI/API credentials and upload automation keys are represented only by safe state such as Configured, Not configured, or Specialized page only. The central Settings feature requires no new database migration and does not rename or duplicate existing keys.

14.3 Gallery management

Gallery administration covers the full life of a collection: creation, discovery, naming, description, dates, tags, privacy, passwords, branding, covers, layout, ordering, moving, and eventual deletion. A child gallery can inherit useful choices from its parent. For example, every gallery inside a private family branch can follow the parent's protection, and an individual child can use its own presentation choice where the editor offers an override.

14.4 Image management

Photographs can be uploaded in the browser, copied into folders and discovered, received through configured automation, transferred from another compatible installation, or added through a scoped mobile workflow. After a photograph arrives, the gallery records its size and available camera information and prepares the smaller display copies used throughout the site.

Image tools include:

- title, description, visibility, tags, and ordering;
- EXIF-derived information and date suggestions;
- GPS visibility and maps;
- bulk selection, move, copy, download, and deletion;
- public voting and score state;
- duplicate review and cleanup.

14.5 Uploads and discovery

Filesystem discovery is useful when photographs have already been copied to the server through FTP, a hosting file manager, or a restored backup. The discovery screen shows what the application found and lets the administrator bring those folders and files into the catalog.

Browser upload is the friendliest choice for routine work. Select the destination, choose files, and follow the progress display. Large sets can be prepared in bounded groups so the browser and server remain responsive. The application checks the destination and file information before accepting each result.

When browser-assisted upload is enabled, the chooser also accepts ZIP archives such as an iCloud Photos export. The browser inspects the archive locally, adds supported JPEG, PNG, WebP, and GIF images to the ordinary preparation queue, and skips folders, hidden metadata, and unsupported entries. The selected ZIP itself is never uploaded to PHP. Stored and Deflate archives are supported; encrypted, multi-disk, ZIP64, damaged, suspiciously compressed, or excessively

large archives fail safely before any server-side upload begins. Keep browser-assisted upload checked when selecting a ZIP because this convenience is intentionally unavailable in the classic PHP upload path.

15 Thumbnail generation and public rendering

15.1 What a thumbnail is

A thumbnail is a smaller display copy of a photograph. The original photograph may be several thousand pixels wide and occupy many megabytes. A gallery card on a phone may need only a few hundred pixels. Sending the full original for every small card would make visitors wait for detail they cannot see at that size.[13]

Imagine a gallery containing 60 photographs, each with a 12-megabyte original file. Loading every original for the first grid could require hundreds of megabytes. Small thumbnails can reduce that first viewing task dramatically. The original remains available for authorized full viewing or download, while the grid receives efficient previews.[14]

Thumbnails serve several everyday purposes:

- gallery grids appear sooner;
- scrolling uses less memory and network capacity;
- mobile visitors receive an image closer to the size of their screen;
- gallery covers and search results remain compact;
- the large viewer can begin with a suitable preview while a larger source is prepared;
- repeated visits can reuse files already stored in the browser cache.[15]

15.2 Why several thumbnail sizes exist

One small size cannot suit every screen. A narrow phone card may look clear with a 300-pixel thumbnail. A wide desktop card may need 600 or 800 pixels. A high-density screen uses more image pixels to draw the same physical area and may benefit from a larger candidate.

PHP Gallery can prepare common sizes such as 300, 600, 800, 960, 1280, and 1600 pixels, depending on configuration and source-image limits. The number describes the maximum long side of the generated copy. Portrait and landscape photographs keep their original proportions.[16]

The smaller candidates serve grids and covers. Larger candidates serve wide cards, high-density displays, search presentations, map previews, and the large viewer. The browser can select from the available group instead of receiving one fixed size for every situation.

15.3 Why JPEG and WebP are available

JPEG is widely compatible and familiar for photographs. WebP often provides similar visual quality with fewer transferred bytes. PHP Gallery can generate both so a compatible browser can

[13] Thumbnail generation keeps the source photograph as the archival and full-viewing asset. The generated copy is a replaceable browsing aid.

[14] Actual savings depend on image content, quality, format, dimensions, and the browser's selected candidate. The example illustrates scale rather than promising one fixed compression ratio.

[15] A browser cache stores recently used web resources on the visitor's device according to server and browser policy. Reuse can make a later visit feel much quicker.

[16] For a landscape photograph, the long side is usually the width. For a portrait photograph, it is usually the height. The shorter side is calculated from the original aspect ratio.

use WebP while JPEG remains a dependable alternative.[17]

The visitor does not choose a format manually. The page tells the browser which versions exist, and the browser chooses a format it understands. The original photograph keeps its own source format and remains separate from these browsing copies.

15.4 When thumbnails are created

Thumbnails can be prepared when photographs are imported or uploaded, rebuilt through Admin maintenance, or generated through a guarded repair process when the public page discovers that an expected copy is missing. Preparing them near upload time gives later visitors the smoothest experience because the required files already exist before the first visit.[18]

A gallery owner should review thumbnail maintenance after:

- importing a large existing folder tree;
- changing enabled thumbnail sizes or quality;
- restoring a backup that contains originals and database data but an incomplete thumbnail cache;
- moving an installation between servers with different image-format support;
- seeing repeated original-file fallbacks or missing previews in System Health.

Generated thumbnails can be recreated from the source photographs. This makes them suitable for cache and maintenance workflows. Source photographs deserve permanent backup protection.

15.5 What visitors receive

The gallery sends the browser a list of thumbnail copies that really exist for a card. The browser considers the space available on the page, the sharpness needs of the screen, the supported format, and its own loading decisions. It then requests an appropriate file.

For example, suppose a photograph has 300, 600, 800, and 960-pixel copies:

- a small phone card may receive the 300-pixel copy;
- a medium card may receive the 600-pixel copy;
- a large or high-density card may receive the 800 or 960-pixel copy;
- the large viewer may request a still larger preview prepared for that purpose.

The exact choice can differ between browsers and screens. This flexibility is useful because the server does not need to guess one universal size.

[17] Compression efficiency varies with the photograph and quality settings. Fine texture, noise, gradients, and sharp graphic details can produce different results.

[18] Large imports can require meaningful processing time and storage. Running preparation as an intentional Admin task gives the owner a clearer opportunity to monitor completion.

15.6 Responsive browser selection

Responsive browser selection is the default mode under **Admin > Theme > Layout**. It gives the browser the full list of suitable available thumbnails as soon as the page is read. The browser can begin requesting the size it considers best without waiting for an additional gallery script.[19]

This mode is a strong general-purpose choice. It usually transfers one selected thumbnail for each loaded card. It works fully when JavaScript is unavailable, and modern browsers have extensive experience choosing responsive images.

15.6.1 Example of responsive selection

Anna opens a six-column gallery on a large desktop display. Each card is fairly narrow, so the browser may select a medium thumbnail. She rotates a tablet where cards become wider, and that browser may select a larger copy. The gallery owner maintains one set of photographs and thumbnails while each visitor receives a suitable presentation.

The 300-pixel copy acts as a practical fallback when available. Its presence does not force a modern browser to download it first because the larger choices are presented immediately.

15.7 Progressive thumbnail sharpening

Progressive thumbnail sharpening is an optional mode under **Admin > Theme > Layout**. It emphasizes the feeling that the page is ready quickly. The gallery first presents a small thumbnail, usually 300 pixels, and sharpens relevant photographs later with larger copies.

This can feel pleasant on a slow connection because the visitor sees the collection taking shape before every card reaches its final sharpness. The page reserves the correct spaces for photographs, so the layout remains stable while images improve.

15.7.1 What happens on the screen

The visitor experience follows this sequence:

1. The page appears with correctly sized photo spaces.
2. Small thumbnails begin filling the visible cards.
3. The visitor can read, scroll, open links, and use the page.
4. Photographs on or near the screen join a short sharpening queue.
5. The gallery chooses a larger copy suitable for each card.
6. A larger copy loads quietly while the small one remains visible.
7. The card becomes sharper after the replacement is ready.

Only a small number of larger copies are prepared at once. Visible cards receive attention before cards farther down the page. This prevents a large gallery from starting dozens of sharpening

[19] The underlying web standard calls this candidate list a *srcset*. Gallery owners can rely on the visual behavior without learning that markup term.

downloads together.[20]

15.7.2 The transfer tradeoff

Progressive mode can transfer two files for one photograph: the small first copy and the sharper replacement. Responsive mode often transfers one browser-selected copy. Progressive mode therefore favors early visual response, while responsive mode often favors lower total transfer for the same card.

> **Performance note.** Progressive rendering prioritizes early visual population and page responsiveness. A session can transfer the small thumbnail and a larger replacement. Responsive rendering commonly minimizes that deliberate two-stage transfer by exposing all candidates immediately.

The current progressive choice applies to photo cards inside an open gallery. Gallery covers, collage cells, home-page gallery cards, and Admin images continue using responsive browser selection. The Admin interface currently labels progressive sharpening as Beta so owners know that the option is receiving practical evaluation.[21]

15.8 Which mode should an owner choose?

Choose **Responsive browser selection** for a dependable default, efficient browser-native choice, and typically one thumbnail transfer per loaded card.

Try **Progressive thumbnail sharpening** when early page response matters strongly, many visitors use slow connections, and a small-first visual experience suits the gallery. Test it with representative galleries and devices before making it the normal presentation.

Useful questions include:

- Do visitors see the first photographs sooner?
- Does the small version look acceptable during sharpening?
- How much extra data is transferred on a typical visit?
- Do phones and high-density screens sharpen at a comfortable pace?
- Does the chosen mode suit the audience's likely connection quality?

15.9 Loading priorities and stable layout

The first visible photographs receive earlier loading attention because they shape the visitor's first impression. Photographs farther down the page use lazy loading, which means the browser can wait until they approach the visible area. This avoids spending connection capacity on the bottom of a long gallery while the visitor is still looking at the top.[22]

[20] The current progressive scheduler permits two larger-image preparation jobs at a time. This bounded queue preserves capacity for interaction and other page resources.

[21] The Beta label describes the maturity shown to administrators. The saved choice and feature implementation use the permanent name *progressive*.

[22] Lazy loading timing belongs partly to the browser. Browsers consider scrolling, viewport distance, connection state, and their own implementation policy.

The gallery knows the width and height of indexed photographs. It uses those proportions to reserve stable card shapes before image loading finishes. Text, buttons, and neighboring cards therefore keep their positions as thumbnails appear.

Visitors who prefer reduced motion receive a calm presentation without a required continuous loading animation. Photograph links remain usable throughout loading and sharpening.

15.10 Thumbnail quality and storage

Higher quality preserves more fine detail and can create larger files. Lower quality saves space and transfer while introducing more visible compression. The best setting depends on subject matter, screen use, hosting capacity, and audience. Detailed foliage, hair, text, and fine architecture can reveal compression more readily than broad soft backgrounds.

Every additional enabled size and format uses storage because it creates another copy for each photograph. The benefit is more precise delivery to different screens. System Health and storage statistics help an owner understand the resulting cache. A large source archive may produce a substantial thumbnail collection, yet those files remain reproducible from the originals.

15.11 If a thumbnail is missing

A missing thumbnail may appear as a slower original-file fallback, an empty preview, or a maintenance warning, depending on the image and available files. Open the thumbnail maintenance tools, inspect the affected gallery or sizes, and rebuild the missing variants. Review write permissions and image-format support when rebuilding repeatedly fails.

After repair, open the gallery with an empty browser cache and confirm that covers, cards, and the large viewer receive the expected previews.

16 Lightbox, maps, EXIF, and voting

16.1 Asynchronous lightbox metadata

The lightbox is the large photograph viewer that opens above the gallery page. It lets visitors concentrate on one photograph while retaining forward, backward, close, full-screen, slideshow, map, and other available controls.

The gallery prepares the viewer when it becomes useful and obtains information for nearby photographs in manageable groups. A visitor can therefore move through a large gallery without requiring the first page to carry every title, preview, map point, and vote at once.[23]

The viewer usually starts with an appropriately sized preview. A larger or original source can be used where the action and access policy call for it. Nearby previews may be prepared quietly so the next photograph appears smoothly.

16.2 EXIF and map policy

EXIF is information many cameras and phones save inside a photograph. It can include capture date, camera, lens, exposure, dimensions, orientation, and GPS coordinates. PHP Gallery can use this information for date suggestions, maps, technical details, ordering assistance, and duplicate review.

Maps load when a visitor asks to see them, which saves work for visitors interested only in the photographs. A gallery can show individual photo locations and an overview of available points. Parent and child settings determine which galleries expose this information.

> **Security note.** GPS data can reveal sensitive locations. Administrators should review inherited map settings, image metadata, and public access before publishing personal photographs.

16.3 Voting

Public cards and lightbox controls share vote state. Server-rendered forms provide direct behavior, while JavaScript submits normal interactions asynchronously and synchronizes the score across matching views. The server validates image identity, gallery policy, and request integrity.

[23] The current viewer normally requests information in bounded windows of nearby photographs. Access checks are repeated for each request so private gallery rules continue to apply.

17 Duplicate Photo Detector

The Admin Duplicate Photo Detector analyzes a selected gallery branch by default and can expand to global scope when the administrator enables the corresponding option. It reports exact and possible duplicate pairs, provides gallery and photo context links, records review decisions, and delegates confirmed deletion to the normal gallery image-deletion service.[24]

17.1 Evidence categories

17.1.1 Exact and possible findings

An exact duplicate contains the same file content. The application recognizes this through a checksum, which acts like a very detailed digital fingerprint calculated from the file. Matching fingerprints provide strong evidence that two files contain the same bytes.

A possible duplicate has supporting similarities without an exact fingerprint match. The detector can compare file size and available camera information such as dimensions, capture time, camera, lens, exposure, aperture, focal length, ISO, and orientation. Two separately exported copies of one photograph may differ as files while retaining enough shared information to deserve review.

Missing camera information simply gives the detector fewer clues. The results remain suggestions for a person to inspect. Open both photograph links, compare their gallery context and quality, and delete only the copy whose removal fits the collection.

17.2 Persistent review ledger

The ledger belongs to the authenticated administrator. Pair entries store canonical image identifiers, with the lower identifier in the low column and the higher identifier in the high column. Gallery entries apply to the exact stored gallery. Parent and child gallery decisions remain independent.

Available review actions include:

- ignore the selected pair;
- ignore all findings from the left image's exact gallery;
- ignore all findings from the right image's exact gallery;
- clear the current administrator's ledger;
- delete a confirmed duplicate through validated normal image deletion.

AJAX actions refresh the detector fragment in the existing Admin right-side panel. Direct POST requests use redirect behavior as a fallback. Server-owned scan jobs preserve scope between continuation requests, and deletion prunes the affected image from current job results.

[24] The detector remains a review tool. Evidence supports administrator judgment because matching file size and photographic metadata can describe distinct photographs under some workflows.

18 Access control and security

18.1 Layered access evaluation

18.1.1 Request authorization

Privacy is decided in layers. The gallery has a visibility and access choice, its parent can provide inherited protection, an individual photograph has its own visibility, and age-restricted content can require confirmation. Password and share-link state determine whether the current visitor has completed the required access step.

These rules also apply when the browser requests a thumbnail, map point, large preview, or original file. A copied image address therefore continues through the same access checks as the gallery page.

Admin tools require a signed-in account. Forms include a private request token that helps the application distinguish an intended Admin action from a forged submission. The server also checks the selected gallery, photograph, and operation before changing data.[25]

18.2 Passwords and share links

Protected galleries store password hashes and grant session-scoped access after successful verification. Share links use server-generated token state and expiry policy. Child-gallery access resolution follows the effective inherited rules represented by the local services.

Password reset uses one-time hashed tokens, optional recovery email, configured lifetime, and selectable mail transport. Administrators should use HTTPS, strong credentials, restricted database accounts, and protected mail secrets.

18.3 NSFW Guard and temporary database problems

NSFW Guard lets an administrator mark a complete gallery or one photograph as adult material. A gallery setting also protects its child galleries through the normal inherited access rules. Visitors receive an age-confirmation step before protected photographs, thumbnails, map information, previews, and original files are supplied.[26]

The protection choices live in two database fields, one for galleries and one for photographs. The application checks both fields before it relies on them. Three clear outcomes help the owner respond correctly:

Protection is ready. Both fields are present. Existing gallery inheritance, individual photograph settings, and visitor confirmation work normally.

An older database is confirmed. The database answered successfully and confirmed that a protection field is absent. This describes an installation from before NSFW Guard, or an incomplete update. System Health directs the owner to pending migrations. Such an older database could not have stored an NSFW choice, so the application preserves its established pre-feature behavior until the migration is applied.

[25] The technical name for the private form token is a CSRF token. Gallery owners usually interact with it automatically through normal Admin forms.

[26] The age confirmation is an application access control. Gallery owners remain responsible for local law, hosting terms, content classification, and any stronger identity or age-verification duties that apply to their publication.

The answer is unknown. The database could not provide a trustworthy answer. Public requests that could reveal protected photographs or related information receive a temporary service-unavailable response. Admin controls explain the inspection problem, and setting changes wait for a reliable answer.

This temporary refusal protects a collection during a database outage, permission change, server move, or incorrect database selection. The visitor sees a short translated message and may see a reference identifier. The message contains no database address, SQL statement, credential, private path, or photograph metadata. Image and thumbnail addresses receive plain text, while lightbox, map, and search requests receive structured JSON so the browser can handle the interruption cleanly.

At the same moment, the application makes one bounded operational log entry with a stable security event name. It records the affected feature, the unknown readiness state, the requested route, the response type, and the same request reference shown to the visitor.[27] This gives an administrator a useful correlation point while keeping sensitive connection details out of the public response. If the Admin log table is also unavailable, the gallery still returns the protective response normally.

An owner can resolve the warning with the following sequence:

1. Open Admin System Health and read the NSFW Guard database card.
2. Note whether the card reports a confirmed missing migration or an inspection failure.
3. For an inspection failure, verify the database server, selected database, application credentials, and permission to read database metadata.
4. For confirmed missing fields, create a backup and apply the pending migrations through the normal maintenance workflow.
5. Reload System Health and confirm that the card reports the protection fields as installed and verified.
6. Open one protected gallery as an anonymous visitor and confirm the age prompt, thumbnail protection, lightbox behavior, map behavior, and original media protection.

The Admin dashboard brings attention to this condition before the detailed card is opened. A confirmed missing field or an unknown database answer places an **Action** badge on Maintenance and System Health. The System Health card and the separate Runtime Diagnostics page use the same status information, so their explanation, affected database-field names, suggested checks, and request reference remain consistent. Runtime Diagnostics also places this bounded information in its copyable report for an owner who needs help from a hosting provider.[28]

The health vocabulary can also describe a feature that an owner intentionally disabled in configuration. NSFW Guard currently remains a core access policy without such an on/off feature switch, so its normal statuses are ready, confirmed older database, and unknown. This distinction keeps an operational database problem separate from a deliberate configuration choice.

Gallery, photograph, and bulk NSFW controls accept changes only while both fields are verified. Other unrelated editing can continue where its own data is healthy. This keeps a temporary database inspection problem from silently clearing or weakening an existing protection choice.

[27] The event name is `security.nsfw_schema_inspection_unavailable`. Its context uses a fixed diagnostic category and contains no SQL statement, exception text, credential, token, or private filesystem path.

[28] Affected-object labels are accepted only when they are valid database table and field identifiers. Database error text and malformed names are excluded from the Admin health model.

18.4 Password, share-link, and administrator database readiness

The same database-readiness protection now covers the other access and account features that can authorize a visitor or administrator. This is important during an interrupted update: seeing that "something is missing" is not enough to justify a permissive fallback. The gallery first distinguishes a confirmed older database from a database whose structure cannot currently be inspected.

Admin **System Health** and **Runtime Diagnostics** show these security areas separately:

- gallery password and access policy;
- gallery visibility compatibility;
- gallery share-link storage;
- NSFW Guard;
- persistent administrator login;
- password-reset storage; and
- external identity links, currently used by optional Google login.

A green or ready result means the required database objects were verified. A confirmed missing result means the database answered successfully and the owner should normally apply a pending migration. An unknown result means the application could not obtain a trustworthy answer, for example because the database is unreachable, the wrong database is selected, or metadata inspection permission changed. Security-sensitive behavior does not treat that unknown answer as if the feature had never existed.

18.4.1 Gallery passwords and interrupted migrations

Password protection uses several related fields rather than one switch. They record the access mode, whether the gallery is listed, the password hash, the share-link validating hash, and its optional expiry. The historical pre-password-protection database contained none of those fields.

For compatibility, PHP Gallery therefore considers the database genuinely legacy only when *all* core access fields are successfully inspected and confirmed absent. If some are present and another is missing, the database is partially migrated. The public gallery does not replace that mixed state with "normal" and "listed", because an existing row may already contain a password, unlisted choice, or share-link hash in one of the fields that is present. Public pages and protected media wait for the migration to be repaired.

This check happens before sensitive route handlers can send gallery HTML, thumbnail bytes, original or preview media, lazy lightbox/map metadata, search results, sitemap information, gallery assets, or a protected gallery download. Visitors receive the same minimal temporary-service response described for NSFW Guard, with the response format matched to the route. The controller that would have produced the protected content is not allowed to begin first.

18.4.2 Public, unpublished, and the older draft value

Older PHP Gallery databases used the word `draft` where the current interface uses `unpublished`. The application now verifies the actual database field definition before choosing between those storage words. If the database successfully confirms the historical definition, saving an unpublished

gallery uses the old `draft` value for compatibility. If the definition cannot be inspected, the gallery does not guess which vocabulary is active.

This distinction matters because an SQL failure and a historical enum definition can otherwise look identical to code that asks only a yes/no question. The three-state check keeps a database outage from changing publication behavior.

18.4.3 Share links and safe revocation

A share link has a secret value given to the visitor and a validating hash kept by the gallery. Current installations can also keep an encrypted copy used by Admin to display the existing link. Creating or using a current share link requires the relevant storage to be verified.

Revocation has a deliberately different safety rule. Revoking access makes the site more restrictive, not less restrictive. Once the core validating hash and expiry fields are verified, PHP Gallery can clear that hash even if the optional encrypted display-token field is missing or cannot be inspected. Clearing the hash invalidates every external copy of the link. A database problem therefore cannot make an existing share link impossible to revoke merely because Admin cannot inspect the optional display copy.

18.4.4 Administrator login and "Keep me signed in"

The ordinary authenticated PHP session and the optional persistent browser login are separate layers. The persistent option uses a database table of hashed remember-login tokens. If that table is successfully confirmed absent on an older installation, password login still works for the current PHP session; the site simply cannot issue the longer-lived browser token until the migration is applied.

If the table's readiness is unknown, PHP Gallery does not issue or trust a new persistent token. A browser restoration attempt is refused safely. When a normal password or Google login has already established the PHP session, a temporary problem with the optional remember-token table does not discard that successful session. Admin can continue in the current session and System Health explains why persistent login could not be established.

18.4.5 Recovery email and password reset

The recovery-email field was added after the original username/password account model. A database that successfully confirms the email field is absent can still support username login. Account editing therefore keeps username and password work available on that proven older schema while recovery-email controls remain unavailable.

Password reset is stricter because it needs both the recovery-email field and the one-time reset-token table. Issuing or consuming a reset link requires both to be verified. If either is confirmed missing, Admin receives migration guidance. If the structure is unknown, the reset workflow reports a temporary storage problem instead of pretending that the account or token is invalid. This also prevents a metadata outage from being counted as a bad password attempt.

Reset secrets remain one-time credentials. Visible diagnostics and Admin logs do not contain the reset token, its URL secret, a password, mail password, cookie, CSRF token, raw SQL statement, database connection string, or raw database exception message.

18.4.6 Google identity links

Google login has two independent readiness questions. First, the site needs its OAuth client configuration. Second, the database must be able to store and look up the link between a Google identity and a local Admin account. A complete OAuth configuration does not prove that the identity-link table is healthy.

A confirmed missing identity-link table is shown as a migration requirement. An unknown table state temporarily disables linked-account lookup, Google login, linking, and disconnect operations until inspection is reliable. Password login remains independent. Callback and schema-failure logs use bounded categories and identifiers rather than copying raw external or database exception text.

18.4.7 What the owner should do

When any security/authentication card shows an action state:

1. Open **System Health** and identify whether the state is confirmed missing or unknown.
2. If it is unknown, check database reachability, selected database, and metadata-inspection permissions before applying migrations repeatedly.
3. If it is confirmed missing, make a backup and apply the pending migration through the normal maintenance workflow.
4. Reload System Health. A migration performed in the same Admin or command process clears the request-local readiness cache after schema changes, so the new structure can be inspected immediately.
5. Re-test the affected behavior. For gallery access, include an anonymous password-protected gallery, an unlisted gallery, a share link, a thumbnail, original media, and gallery download. For account storage, test ordinary login, persistent login if enabled, password reset if configured, and Google login if used.

The copyable Runtime Diagnostics report uses the same bounded health model as System Health. It may include validated table or field names, a safe status, suggested checks, and a request reference. It does not include the private data that caused or accompanied a database exception.

18.5 Destructive operations, uploads, and database readiness

Some database checks protect more than whether a page can be displayed. Deleting a gallery, moving a photograph, accepting an upload, creating an upload key, installing migration files, rebuilding thumbnail metadata, cleaning a database, or activating an application update can change information that is difficult to reconstruct after only half of the operation succeeds. PHP Gallery therefore uses the same three readiness states for these data-changing workflows, but applies a stricter rule than it can use for optional visual features.

The three states mean:

Available. The database confirmed the tables and fields required by the operation. The existing workflow continues normally.

Missing. The database inspection worked and positively confirmed that a required item is absent.

Usually this means a pending migration. A small number of workflows have an explicitly supported older-database or migration-bootstrap mode, which is described below.

Unknown. PHP Gallery could not obtain a trustworthy answer about the required database structure. The affected data-changing operation is paused before its target is changed. The application does not interpret an inspection error as proof that the table or field is absent.

This distinction is especially useful on shared hosting. A temporary database outage, a changed metadata permission, or accidentally selecting the wrong database can make an old yes/no existence check look exactly like a genuinely missing field. For a deletion or upload those two situations must not be treated the same. Confirmed absence can lead the owner to a migration. Unknown readiness means the database connection or inspection problem should be solved first.

18.5.1 Which operations are protected

Admin **System Health** reports separate readiness for the Phase 10 workflows:

- gallery and photograph deletion;
- gallery and photograph move/copy operations;
- the per-administrator Duplicate Photo Detector review ledger;
- normal browser uploads and browser-prepared ZIP uploads;
- gallery-scoped upload automation API keys;
- gallery migration/import jobs;
- mobile WebDAV upload credentials and uploads;
- thumbnail metadata generation and maintenance;
- database cleanup and schema repair;
- application update activation.

A **Missing** or **Unknown** state contributes to the **Action** badge on Maintenance. Runtime Diagnostics shows the same capability list in a separate destructive/ingestion section. The two screens intentionally use the same database readiness model, so an owner should not see one page claim a workflow is healthy while the other reports it as uninspectable.

18.5.2 The safety check happens before the target changes

The important protection is the order of events. The application checks the database shape before the first irreversible change to the operation's target. Examples include:

- A classic upload is checked before PHP moves the uploaded source file from temporary storage into the gallery folder.
- A browser-prepared ZIP is checked before an original image from the package is written into the gallery.
- A WebDAV PUT checks both credential storage and gallery-ingestion storage before the request becomes a gallery file.

- A gallery migration checks the target schema before it writes imported metadata, installs originals or thumbnails, or completes the job.
- Thumbnail generation checks the metadata write shape before creating or replacing derivative files.
- Gallery deletion and move/copy operations check ownership and path fields before database or filesystem changes begin.
- Database cleanup and repair require a conclusive schema-history check before live mutation.
- An application update may already have been downloaded and extracted into staging, but the final activation check runs immediately before active application files are replaced.

The purpose is recoverability. When readiness is unknown, an original gallery file should remain where it was, a prepared upload should not become a half-indexed gallery item, a migration job should remain resumable, an existing credential should remain unchanged, and the currently running application should not be partially replaced by an update.

18.5.3 Confirmed older schemas and thumbnail compatibility

Not every confirmed missing item means data must be destroyed or the whole site must stop. Some compatibility behavior is safe only because the database has positively answered that the newer object is absent.

Thumbnail metadata is one example. Older installations can have generated thumbnail files without the newer `image_thumbnail_variants` metadata table. If the table is confirmed absent, PHP Gallery can preserve its documented file-only thumbnail compatibility. If the table exists, the application also checks optional fields used by the newer metadata write format before generating, repairing, replacing, or deleting derivatives. An inspection failure at that point does not fall back to file-only mode, because the table may actually exist and contain authoritative state.

Gallery migration uses the same principle for optional imported metadata. A field that is confirmed absent may be omitted from an older target database. A field that cannot be inspected is not silently discarded. The migration pauses so the owner does not receive an apparently successful import with an unpredictable subset of metadata.

Database maintenance and application update activation have a different documented bootstrap case. If the database positively confirms that `schema_migrations` is absent, the migration system can create or initialize its own history table as part of the established workflow. If the existence of that table is unknown, cleanup, repair, or active-file replacement waits for reliable inspection instead.

18.5.4 Upload keys and WebDAV revocation

Creating or trusting an upload credential requires the complete database shape used by that authentication method. PHP Gallery must be able to verify the token hash and the other fields needed by the existing authentication workflow before it creates or accepts the credential.

Revocation is intentionally narrower because it makes access more restrictive. If the smaller set of fields needed to disable an upload-automation key is verified, the administrator can revoke that key even when an unrelated token field is missing. Mobile WebDAV credential deletion follows the same principle with the verified row identity used by the delete operation. This is not permission

to guess through an inspection outage: if the fields needed for revocation themselves cannot be inspected, the revocation operation is also paused.

This asymmetry avoids an undesirable result in which an incomplete but inspectable upgrade prevents the owner from shutting off an existing external upload path.

18.5.5 What the owner sees when an operation is paused

The exact screen depends on the workflow. An Admin page may show a migration-required message, a temporary database-schema warning, or a normal failed-operation notice. JSON-based side panels and upload managers can return a conflict or temporary-service status appropriate to the cause. WebDAV and public automation endpoints return a temporary failure instead of pretending an inspection outage is simply an invalid credential.

The diagnostic information is deliberately limited. System Health and Runtime Diagnostics can show a capability name, **Available**, **Missing**, or **Unknown**, validated table/field names, suggested checks, and a request reference. The mutation refusal log uses the same bounded identifiers. These surfaces do not show raw SQL, raw database exception text, DSNs, passwords, API keys, WebDAV passwords, uploaded private paths, migration source paths, or update staging paths.

18.5.6 Recommended recovery procedure

If a deletion, upload, migration, thumbnail operation, database repair, or update is paused because its schema is unknown:

1. Do not repeatedly retry a destructive operation while the database state is still unclear.
2. Open **System Health** and identify the exact mutation capability that is **Unknown**.
3. Check normal database connectivity, the configured/selected database, and whether the application account can read database metadata.
4. Check the Admin log using the visible request reference when one is shown. The useful entry should describe the bounded capability and operation, not expose private database details.
5. Restore reliable database inspection and reload System Health.
6. If the state becomes **Missing**, make a coordinated backup and apply the pending migration through the normal maintenance workflow. Do not use repeated migrations as the first response to an **Unknown** state.
7. If the state becomes **Available**, retry the original operation and verify both its filesystem result and its database result.
8. For a gallery migration, resume the existing job instead of starting a second import when the first job remains available.
9. For an update that was downloaded but not activated, re-run the normal update action after readiness is healthy. The staged package can be replaced or reused by the normal updater workflow; the important point is that the active application was not partially copied during the refusal.

For high-value galleries, test these safeguards on a disposable copy after a major server move or database restore. A useful manual exercise is to temporarily remove metadata-inspection

permission, attempt one upload and one deletion, and verify that the target gallery remains unchanged while System Health reports **Unknown**. Then restore permission, confirm the state becomes **Available**, and repeat the normal operation.

18.6 Optional maps, games, AI, and reporting database readiness

Not every database-backed feature controls access or deletes data. Some features add information or interaction around the main gallery: a GPS map, a stored flight route, voting, Picture Game, a special lightbox mode, AI-assisted text, internal AI search metadata, navigation data, telemetry, or a detailed administrator report. PHP Gallery treats these as optional presentation and reporting capabilities.

The important owner-facing rule is that a safe optional display failure does not have to take the whole gallery offline. If the database cannot verify the GPS map fields, for example, the main gallery can still show its photographs without the map. Omitting the map does not reveal a protected photograph and does not grant a visitor any additional permission. System Health still records the map capability as **Unknown**, so the problem is visible instead of being silently mistaken for a feature that was never installed.

A database-backed *write* is different. Recording a vote, remembering which two photographs Picture Game displayed, saving a per-gallery lightbox or GPS override, enabling voting while creating or importing a gallery, carrying an inherited lightbox choice into an organizer-created child gallery, changing an OpenAI profile, changing an AI analysis job, storing navigation account tokens, or running telemetry maintenance changes persistent state. An **Unknown** database shape is never permission to perform those writes or silently discard the requested optional value.

18.6.1 The four states shown in System Health

Optional capabilities can show four states:

Available. The required database objects were verified and the optional feature can use its normal behavior.

Missing. The database was inspected successfully and the optional object is genuinely absent. The feature may be hidden or may ask for a pending migration. Some explicitly supported old installations can continue without storing the newer optional information.

Unknown. PHP Gallery could not verify the database shape. A safe visual surface may be omitted, but dependent writes are paused and the problem appears in diagnostics.

Disabled. The optional feature is intentionally turned off by configuration. System Health reports that fact directly. It does not query the database merely to inspect a feature that cannot affect the current request.

The **Disabled** state is also a performance feature. The System Health registry checks the real feature flag before asking about its tables and fields. Enabled features reuse the same request-local database-readiness cache described later in this manual.

18.6.2 Maps and lightbox overrides

GPS/EXIF presentation verifies the gallery switch and the image metadata used by the map. Metadata-organizer capture-date grouping uses the same structured inspection model for the

narrower `images.exif\_taken\_at` requirement instead of an older yes/no column probe. The per-gallery GPS choice is a true inherited setting: a database value of `NULL` means that the gallery follows the global/default choice. PHP Gallery therefore verifies that the field is actually nullable before it treats this inheritance state as available. If nullability cannot be inspected, the gallery can still omit the optional map, but Admin does not save a guessed override.

Flight maps use their own route and navigation-data storage. A missing or unknown route map can be omitted from an otherwise usable gallery. Saving or deleting a stored route requires the appropriate database shape to be known. The same principle applies to a per-gallery lightbox browsing-mode override: public viewing may use the safe global/default mode, while an Admin-submitted override waits until the stored field and supported mode vocabulary can be verified. The same protection applies when a gallery is created from `gallery.json` or when the metadata organizer creates a child gallery. A confirmed old database may omit the unavailable optional column, but an inspection outage does not silently throw away an explicit or inherited lightbox choice. Likewise, a new or imported gallery is not allowed to switch voting on until the vote-storage capability is verified.

18.6.3 Voting and Picture Game

Voting and Picture Game are optional visitor interactions, but both write state. A confirmed missing migration can hide the feature or show migration guidance. An inspection failure is not treated as an empty vote table.

Picture Game has one subtle write boundary: when PHP Gallery chooses a pair of photographs to display, it records that pair before the visitor chooses a winner. Consequently, opening the next game pair is not a purely read-only operation. If the Picture Game schema is unknown, the main gallery remains usable but the game route returns a temporary unavailable state and no pair is recorded. Bulk Admin toggles use this exact Picture Game capability instead of an unrelated all-features-ready check.

The Complete Admin Gallery Report also uses the real Picture Game data model. Its statistics distinguish pairs with a recorded `winner_image_id` from pairs that were displayed without a completed selection. It does not query a nonexistent `vote` field in the Picture Game table.

18.6.4 OpenAI and local AI metadata

OpenAI text assistance has a core per-user settings shape and a separate optional field for image input. A confirmed older text-only settings schema can preserve that older behavior. If the settings shape is unknown, PHP Gallery does not save a partial profile while pretending that the missing answer was confirmed.

Local AI image metadata uses result and queue tables. Public/Admin read surfaces may omit optional AI metadata when the database cannot be inspected, but queue claims, heartbeats, completions, retries, and forced reprocessing require a known database shape. The upload-automation companion worker receives a bounded operational error when storage cannot be verified. Raw SQL, a database connection string, or the raw server exception is not returned to the worker.

18.6.5 SimBrief and navigation accounts

SimBrief description generation is intentionally separated from route-map persistence. PHP Gallery can obtain and render a useful SimBrief/OFP-based draft even when the optional flight-

route database storage is absent or temporarily uninspectable. In that situation the draft remains usable, but the application does not claim that the route map was saved.

Navigation data makes a similar distinction between temporary session state and persistent account storage. A database positively confirmed to predate persistent navigation accounts may keep the historical session-only OAuth behavior. An **Unknown** account schema cannot be reinterpreted as that old installation, so new persistent token storage is paused.

Disconnecting an existing navigation account is deliberately narrower. If the row identity and provider ownership needed to remove the stored account are verified, the administrator can still delete that stored credential even when unrelated optional account fields are absent. This mirrors the security principle used for upload-key revocation: making access more restrictive should not require fields that the delete operation does not use.

18.6.6 Telemetry and the Complete Admin Gallery Report

Telemetry has separate database needs for settings and for the complete report and maintenance data set. The telemetry page and HTML export distinguish a confirmed missing migration from an inspection outage. Setting changes require verified settings storage. Rollup and retention cleanup can quietly have nothing to do on a proven pre-telemetry database, but they do not silently report success when the reporting schema is **Unknown**.

The Complete Admin Gallery Report checks its known optional sections through the same structured readiness service. One database query is intentionally different: the report enumerates all base tables in the currently selected database so it can show exact table counts, including future or extension tables whose names the core application does not know in advance. This dynamic inventory uses `information_schema.TABLES` directly. If that inventory fails, the report shows generic unavailable text instead of embedding the raw database exception.

18.6.7 What System Health and Runtime Diagnostics expose

System Health and Runtime Diagnostics use the same fifteen optional capability entries. An **Unknown** entry may name a validated table or field, suggest checking database connectivity, the selected database, or metadata permissions, and include a request reference that helps correlate Admin logs. It does not expose raw SQL, PDO exception messages, DSNs, passwords, OAuth or API tokens, browser cookies, reset or share tokens, CSRF values, or private filesystem paths.

A useful troubleshooting sequence is:

1. Check whether the feature is intentionally **Disabled**.
2. If System Health says **Missing**, review pending migrations and the version of the restored database before changing anything manually.
3. If it says **Unknown**, check database connectivity, the selected database, and permission to inspect metadata before repeatedly applying migrations.
4. Retry the optional feature after readiness becomes **Available**. If it was a write, verify that the earlier failed attempt did not change persistent state.
5. For SimBrief or other split workflows, distinguish the primary result from optional persistence. A generated draft can be valid even when its optional map write was intentionally omitted.

Important. A temporarily missing map, game, AI panel, telemetry report, or optional report section does not by itself prove that a migration disappeared. Read the exact System Health state first. **Missing** means the database positively confirmed absence; **Unknown** means the application could not obtain a trustworthy answer.

18.7 Operational security

Recommended practices include:

- Serve the application through HTTPS.
- Restrict database credentials to the application schema and required operations.
- Keep `config.php`, backups, caches, and generated archives outside public discovery wherever the hosting layout permits.
- Apply migrations and updates through authenticated maintenance workflows.
- Review audit logs and security diagnostics.
- Test protected galleries through anonymous visitor sessions.
- Maintain coordinated filesystem and database backups.

19 Theme, presentation, and localization

The Theme area controls site identity, colors, dimensions, page width, backgrounds, branding, language, gallery-card presentation, lightbox defaults, GPS presentation, favorite navigation, and public thumbnail rendering. Settings use normalized scalar values or focused service models.

19.1 Choosing the interface language

PHP Gallery treats the language of the program interface separately from the text that you write yourself. Changing the interface language changes buttons, menu items, help text, status messages, dialogs, and other application wording. Gallery and photo titles/descriptions can have optional maintained-language versions, but they are edited and saved separately; tags and other owner content remain unchanged.

Open **Theme > Language**. Two independent selectors are available:

- **Admin interface language** changes the language seen by the administrator. The choice is remembered for that administrator in the current browser.
- **Public visitor language** changes the default interface language shown on public gallery pages. This is a site-wide setting for anonymous visitors.

Beside these selections, the reusable **Viewer language selector** panel controls whether public visitors may choose a personal interface language. It appears in both **Theme > Language** and **Settings > General**; both copies edit the same availability settings and therefore always stay synchronized. The feature is enabled by default. The language checklist underneath it enables English, Czech, German, and Swedish by default, and at least one language must remain selected when the panel is saved. This panel does not select a language for the site or for an account: each public viewer makes an independent choice, and that personal choice is saved only in that viewer's browser cookie.

The checklist is site-wide configuration of which choices are offered, but selecting one of those choices affects only the viewer who selects it in that browser. For example, removing Swedish hides the Swedish button and refuses Swedish viewer overrides, but Swedish remains available as the administrator's language, as the site-wide public default, and in the language-pack editor. Disabling the whole viewer feature removes the public language buttons and makes public pages follow **Public visitor language**; saved viewer query parameters, session choices, and cookies are ignored until the feature is enabled again.

These choices do not have to match. For example, an administrator may work in Czech while the public gallery uses English, Swedish, or German.

Every public page includes a compact language control in the shared header. It uses locally bundled SVG artwork for the United Kingdom, Czechia, Germany, and Sweden as visual cues for English, Czech, German, and Swedish. Each control retains the language's native name as its accessible label, so the choice does not depend on recognizing a national flag. Selecting a language keeps the visitor on the same gallery or public page, including applicable filters, and remembers the per-viewer choice in that browser for later visits. The equivalent address parameter, such as `?lang=sv`, remains supported for direct links and no-JavaScript use.

The **Viewer language selector design** editor offers five presets: **Classic**, **Solid pills**, **Outline**, **Soft cards**, and **Minimal**. Classic preserves the original appearance. **Settings > General** shows only the basic preset and flag choices plus a link to the detailed **Theme > Language** editor; saving there preserves every detailed override. Theme uses compact one-line controls

and places the live preview above them. The preview uses the real language labels and bundled flags while flags, codes, native names, layout, active emphasis, colors, padding, margins, gaps, borders, radii, flag size, and text size are adjusted. Every color can use its selected value or explicit transparency. Values are bounded and validated; malformed saved values return safely to canonical defaults. **Reset all language selector settings** selects Classic and restores the complete design, **Reset this preset** restores only the active preset, and each field has its own reset. Resets update the preview but are not persisted until the language form is saved, and they never change the selector feature switch, offered languages, site default, Admin language, or a visitor's browser-local choice.

The flag files are stored under `public/assets/flags/` and are served by the gallery itself; the visitor's browser does not contact an external flag service. The artwork is pinned from `lipis/flag-icons` version 7.2.3 and is distributed under the bundled MIT license in `public/assets/flags/LICENSE.flag-icons.md`.

When the viewer feature is enabled, the menu also provides **Use site default**. This removes the visitor's personal override from both the current session and the browser cookie. The public interface then follows the administrator's **Public visitor language** setting again, including any later change to that site-wide default. This preference also selects matching saved gallery/photo title and description versions when available. It does not change access permissions, generate translations, or alter source content.

19.1.1 Multilingual gallery and photo text

Gallery and photo editors keep the existing title and description as source content. Use **Language of the current title and description** to classify that text as English, Czech, German, Swedish, or leave it **Not specified**. Existing installations are not automatically classified as English.

Open **Other languages** with the globe control to enter optional translated titles and descriptions. Gallery titles and descriptions fall back independently. A saved photo-language row is instead treated as one caption variant: blank fields remain hidden rather than mixing source-language text underneath translated photo text. Leaving both translated fields blank removes that language row. Saving occurs through the normal gallery/photo form and remains in the existing Admin side panel.

Public pages use the viewer's selected maintained language. Matching text is shown on cards, gallery descriptions, photo overlays, lightbox metadata, search results, alt text, and SEO metadata. Missing gallery fields fall back to source content; a present photo caption variant displays only its saved translated fields. Translations do not change URLs, slugs, folders, filenames, ordering, visibility, passwords, NSFW rules, or media authorization.

If OpenAI text assistance is configured, **Suggest translation with OpenAI** sends the source field and inserts a reviewable draft in the target field. The draft is never saved or published automatically. Manual translation remains fully available without a provider.

19.1.2 Available languages and their current state

English is special. It is the canonical source language, the default language for a new installation, and the defensive fallback language. Czech, German, and Swedish are also maintained as complete catalogs. For now, these four languages are the only selectable interface languages. Additional starter files may exist for future translation work, but they are intentionally not offered to administrators or public visitors yet.

Language	Code	Current role
English	`en`	Complete canonical source, new-installation default, and fallback language.
Čeština	`cs`	Complete maintained selectable Czech translation.
Deutsch	`de`	Complete maintained selectable German translation.
Svenska	`sv`	Complete maintained selectable Swedish translation.

Important. Only English, Czech, German, and Swedish are selectable for now. The application still keeps English fallback behavior as a safety mechanism, but the maintained selectable catalogs are expected to stay complete. A future language can be prepared as a file without becoming selectable until it is deliberately added to the supported-language set.

19.1.3 How translation packs work

A translation pack is simply a dictionary of named interface messages. The program asks for a stable message key such as “save gallery” or “delete selected photos”, and the active language pack supplies the wording that should be shown. The internal key stays the same in every language; only the displayed text changes.

For non-technical administration, the important behavior is:

1. English contains the reference wording for the application.
2. Czech, German, and Swedish contain a translated value for every maintained English key.
3. Only English, Czech, German, and Swedish are currently offered in the Admin and Public language selectors.
4. English fallback remains active defensively if a maintained pack ever misses a value.
5. Future language files may be prepared in `app/lang/`, but they stay dormant until deliberately added to the supported-language set.
6. The **Detected language packs** table shows coverage for the currently supported languages.
7. The **Diagnostics** subsection records missing keys seen during an authenticated Admin session, which helps locate remaining translation work.

The **Pack editor** is intended for careful maintenance or for importing a prepared translation. Language data is stored as JSON. In that editor, the text on the left side of each JSON pair is the stable key and should not be translated. The text on the right side is the value that may be translated. Placeholders such as `\{count\}`, `\{gallery\}`, or `\{time\}` represent live values inserted by PHP Gallery and must keep the same names in every language.

A language pack can also be exported as JSON, translated outside PHP Gallery, and imported again. The application validates that the imported file is a JSON object containing text values. This makes it possible to work with a translator without giving that person access to the gallery server.

For developers and maintainers, the maintained catalogs live under `app/lang/`. JSON is the canonical editable format. English, Czech, German, and Swedish PHP dictionaries are retained only as compatibility fallbacks for installations that do not have the corresponding JSON file.

Server-rendered pages and browser JavaScript use the same active-language plus English-fallback model, so a translation does not need to be duplicated in separate front-end and back-end catalogs.

Custom CSS supports installation-specific visual refinement. Theme-generated CSS exposes validated variables and computed selectors through a dedicated route. Gallery branding can override selected site-level assets while preserving effective fallback behavior.

19.2 Gallery hero tags

The **Appearance > Gallery tags** subsection controls the compact tag collection below the title of an open gallery. The tag area uses the complete width of the hero panel, so tags can continue wrapping toward the right edge instead of being limited by the normal readable paragraph width. Direct gallery tags and tags collected from contained content remain separate semantic groups, but each group follows the same global display policy.

The same subsection also controls the public tag landing page. **Galleries per row** and **Rows per page** define the tag-result gallery grid and its page capacity, while **Gallery-card design** selects the vertical or horizontal card presentation for those results. These values affect public pages opened through a tag; they do not change ordinary gallery pages. If the dedicated values have not been saved, the global Theme grid and gallery-card design remain the fallback. The global pagination switch still determines whether a long tag result is split into pages. From **Edit tags**, **Configure tag display** opens this subsection directly, and **Manage tag metadata** provides the reverse contextual link.

By default, the browser initially shows 20 tags. The limit can be changed from 1 to 200 with either a range slider or an exact number field. When more tags exist, **Display all tags** appears and expands the existing tag elements in place with JavaScript; no page reload or additional server request is used. The same control can collapse the collection again. Administrators who do not want progressive disclosure can enable **Display every tag immediately**. All tags are still emitted in the server-rendered HTML in every mode, so disabling JavaScript leaves the complete tag collection visible and usable rather than hiding content permanently.

Tag ordering is configurable as **Most used first** or **Alphabetical**. **Most used first** is the default. Usage is the total number of direct tag assignments to galleries plus direct tag assignments to photographs across the installation. Equal usage counts fall back to a natural case-insensitive alphabetical order and then a stable identifier tie-break. Direct gallery tags are sorted within their group and contained tags are sorted within their group; the groups are not mixed together.

Long tag collections can also use a responsive internal scrollbar. This option is enabled by default and defaults to five visible rows. The row threshold can be configured from 1 to 12 with a slider or exact number field. The browser measures the rows after the tags have actually wrapped at the current hero width, and it enables scrolling only when the measured row count exceeds the configured threshold. Resizing the page causes the measurement to be recalculated. Disabling the scrollbar option removes the height constraint and allows the hero to grow naturally.

These controls are stored as scalar values in the existing application settings table: `theme_hero_tag_visible_lim` `theme_hero_tag_display_all`, `theme_hero_tag_scrollbar_enabled`, `theme_hero_tag_scrollbar_rows`, and `theme_hero_tag_sort_mode`. The tag landing-page controls use `tag_page_gallery_grid_columns`, `tag_page_gallery_grid_rows`, and `tag_page_gallery_description_layout`. They therefore require no database schema migration. Server-side normalization protects the supported numeric ranges, layout values, and sort modes even when a form is submitted without the browser controls.

20 Downloads, sharing, and transfer

Gallery downloads stream authorized content as ZIP archives. Selection downloads package chosen images, while full-gallery operations traverse the permitted scope. Archive paths require normalization to prevent traversal and preserve deterministic names.

Gallery migration exchanges manifests and assets between compatible installations. Resumable status endpoints allow the receiver to report completed work and avoid retransmitting accepted assets after interruption. Browser upload preparation similarly uses manifests and bounded batches, followed by server validation.

Mobile WebDAV tokens associate a user, gallery scope, credential state, and path behavior. Operators should treat these tokens as credentials and revoke unused entries through the corresponding controls.

21 Maintenance, updates, and recovery

21.1 Migrations

Schema changes live in timestamped PHP files under `database/migrations/`. The migration runner prevalidates pending definitions, applies them in filename order, and records successful versions. New schema work receives a new migration file so existing installations retain a deterministic history.[29]

Database-readiness checks are remembered only for the current PHP request. This normally reduces repeated metadata queries, but a migration can change the answer while the same Admin, installer, updater, or command-line process is still running. The migration runner therefore clears those remembered answers after schema-changing statements and after successful repair callbacks. A post-migration check then reads the new database catalog instead of reusing an answer from before the migration. Ordinary data-only migration statements do not clear the readiness cache because they cannot create or remove a table, field, or index.

21.2 How the gallery checks database readiness

Many gallery features need particular places in the database catalog. Password reset needs a secure token table. NSFW Guard needs fields that remember the gallery and photograph protection choices. Upload automation needs a table for hashed upload keys. Thumbnail maintenance needs records that describe generated image sizes. Before using these features, the application can ask the database whether the required table or field is available.

This readiness check protects owners during upgrades. A web host may copy the new application files before the database migration has been applied. A careful feature can recognize the older catalog and guide the administrator toward the pending migration instead of sending a query for a field that has not yet been created. Optional visual features can often use a familiar default while the upgrade is completed. Security and data-changing features deserve stricter handling because their database records participate in protection, ownership, or recovery.

Two situations can look similar on the screen while requiring different solutions:

Confirmed absence. The connection worked, and the database confirmed that a required table, field, or index is absent. This usually points to a pending migration, an interrupted update, or a restored database from an older application version.

Unknown readiness. The application could not obtain a trustworthy answer. Possible causes include a temporary database outage, incorrect connection details, a hosting permission change, the wrong selected database, or a query failure. Repeatedly applying migrations before checking connectivity can obscure the original problem.

The application now contains a shared inspection foundation that records three clear outcomes: *available*, *missing*, and *unknown*. It checks tables, fields, and indexes through database metadata, remembers repeated answers for the current page request, and keeps technical failure details bounded so credentials and private connection information stay out of visible diagnostics.[30]

[29] Some migrations include repair callbacks in addition to SQL. Compatibility tests protect partial-update scenarios where an older runner encounters newer migration definitions.

[30] The repository-wide conversion is complete. NSFW Guard was the first security-sensitive pilot. Gallery access/authentication now use explicit security policy; destructive and ingestion workflows use a stricter mutation policy; and optional maps, voting, Picture Game, lightbox overrides, AI assistance, navigation integrations, telemetry, SimBrief route persistence, and Admin reporting use the final presentation/reporting policy.

When a feature needs several database items, the foundation keeps the result for every item. An unknown answer receives priority over a missing answer because the complete condition could not be verified. This gives future System Health messages enough detail to identify a confirmed pending migration separately from a connection or permission problem.

The first production pilot, NSFW Guard, needs exactly two database fields. The first NSFW readiness check in one page request therefore asks the database catalog about those two fields once each. Later gallery, photograph, thumbnail, lightbox, map, or media checks in the same request reuse the same answers. Phase 9 applies the same request-local reuse to access and authentication capabilities, including the shared recovery-email field used by both login identity lookup and password reset. Another browser request starts with a fresh cache. This keeps the safety check predictable without adding repeated catalog queries to every protected item or authentication screen.

Mutation checks reuse the same cache. For example, the complete gallery upload-ingestion capability needs twelve first-use metadata probes for its two required tables and core columns, while another request for the same capability inside that PHP request adds no further catalog probes. Thumbnail-write preflight can add checks for optional metadata columns when the metadata table exists; those answers are also reused by later thumbnail registration in the same request.

Optional presentation health checks share this cache as well. The image-voting capability has ten first-use table/field requirements and does not repeat them in the same request. The optional health registry is lazy: merely building the registry performs no database metadata queries, and a feature that is switched off reports **Disabled** before its resolver is called.

The readiness foundation has focused safety tests that recreate successful, missing, connection-failure, and permission-failure answers without altering an owner's database. They also check that repeated questions share one answer during the current page request, that a fresh inspection follows successful schema DDL or a safely replayed duplicate DDL operation, and that simulated passwords, tokens, hostnames, and private paths stay outside the diagnostic result.[31]

For an owner, the practical response is straightforward:

1. Open Admin System Health and review database connectivity and pending migrations together.
2. Confirm that the application is connected to the intended database, especially after a server move, restore, or hosting control-panel change.
3. Create a coordinated backup before applying pending migrations.
4. Apply migrations through the normal authenticated maintenance workflow.
5. Reload System Health and verify the affected feature again.
6. If inspection still fails, preserve the exact time, visible reference identifier, application version, and relevant server log entry for the hosting provider or maintainer.

An unknown readiness result deserves a cautious response. Public access and privacy decisions should stop safely, login-related operations should preserve their security controls, and uploads or deletions should wait for a reliable database answer. Optional appearance/reporting controls may use a safe default or omit only the affected surface while the main gallery continues, provided

[31] The tests also verify that table, field, and index lookups remain limited to the database selected by the application's connection, pass object names as prepared-query values, and reset cached readiness after migration repair callbacks. These checks support the permanent security, mutation, and optional-presentation policy layers that now share the foundation.

omission cannot weaken access or authorize a write. Admin diagnostics can still present the portions they could inspect and clearly identify the failed portion. This approach gives the owner useful pages while protecting photographs, accounts, and persistent data.

> **Important.** When a feature suddenly disappears after working normally, check database connectivity and permissions before assuming that its migration vanished. A genuinely missing migration is usually associated with an update, a fresh installation, or restoration of an older database backup.

21.3 Thumbnail maintenance

Maintenance tools inspect generated variants, identify missing or stale derivatives, and rebuild selected sizes. Browser-assisted rebuilding can prepare source chunks where configured. Background warm-up handles small public-rendering gaps with bounded requests, while explicit Admin maintenance remains the primary large-scale repair path.

21.4 Database maintenance

Database inspection inventories schema objects, references, ownership categories, retention policy, and cleanup confidence. Logical cleanup uses allow-listed rules with deterministic selection, bounded batches, and audit records. Schema repair provides a dry-run and idempotent migration path. Physical table optimization remains a separately confirmed selected-table operation.[32]

21.5 Admin log history and archives

Everyday administrative work creates a useful history: uploads, gallery changes, security decisions, maintenance results, warnings, and other operational events can leave a record in the Admin log. That history is valuable when you want to understand what happened, when it happened, and which account or request was involved. It is also normal for the table to become large on an active installation. Version 0.90 therefore treats log care as its own explicit maintenance task rather than allowing generic database cleanup to remove audit history unexpectedly.

The Admin log screen can show individual entries or group repeated events into a compact summary. A group might represent many occurrences of the same kind of thumbnail warning, upload result, or maintenance message. Opening the group reveals its members. This gives an administrator a useful overview without losing the underlying detail. Large lists and exports are processed in bounded portions, so requesting a complete history does not require the browser or PHP to hold the entire table in memory at once.

The owner can choose how long recent logs should remain in the live database. Choosing *Forever* disables automatic archival. Choosing a finite live period does not immediately throw older records away. The maintenance workflow first selects complete calendar days that are safely older than the live window, writes a protected archive containing both machine-readable JSON and a readable static HTML report, checks that the archive contains the expected number of records, and only then removes those same rows from the live table. The archive is stored under `data/admin-log-archives/`, which is application data rather than a public gallery area and carries an additional web-server protection rule.

[32] Storage-engine allocation values such as data-free estimates describe engine state and do not guarantee a precise number of filesystem bytes returned after optimization.

Archive maintenance is deliberately recoverable. It writes temporary files before promoting a completed archive, records a small state file, and uses a lock so two maintenance requests do not process the same history simultaneously. If hosting limits, a connection interruption, or another problem stops the work, the Admin screen can show the incomplete state and the next maintenance attempt can verify, continue, or safely recover it. An archive is never considered a reason to delete live records merely because a file with a similar name exists; the row-count and archive metadata checks are part of the safety boundary.

Administrators should still include the database and the `data/admin-log-archives/` directory in coordinated backups. The live database contains recent operational history, while the archive directory contains older history selected by the explicit retention policy. Keeping both gives the owner a continuous record and makes a restored installation easier to investigate after an update, migration, or unexpected event.

21.6 Application updates

The updater checks configured channels, validates packages, stages content, verifies required runtime files and sizes, preserves overwritten files in recovery storage, and activates the update. Rate-limit handling and retry state prevent repeated remote requests during provider wait periods. A resumable job card appears in both **Status** and **Advanced tools**, so a beta install, restore, or reinstall continues to show its progress where it was started. The page may be closed and reopened without discarding completed checkpoints.

Temporary download or hosting failures may be retried safely. A package whose files do not match its own integrity manifest is different: downloading the same published build again cannot change those bytes. PHP Gallery therefore asks the administrator to cancel that prepared job and choose a newer release or beta code instead of offering an endless retry loop.

> **Important.** Run a coordinated backup before an application update. Keep the backup available until runtime checks, migrations, public galleries, protected access, uploads, and Admin operations have been verified.

21.7 Integrity manifest

`app/core-manifest.json` records the application version and hashes for integrity-managed core files. The local generator under `scripts/generate_manifest.php` calculates final hashes after runtime changes. Documentation and user-data inclusion follow the generator's established policy.

22 Optional background: how PHP Gallery works

This section is optional background for curious owners, hosting administrators, and developers. Daily gallery work can continue without these implementation details. The purpose is to explain the main pieces in approachable terms and provide precise file references for someone maintaining the installation.

22.1 Bootstrap and dispatch

When someone opens a page, the application first reads its configuration and determines which public or Admin action the address represents. It then asks the responsible part of the application to gather information, apply access rules, and produce a page, file, download, or saved result. The central starting file is `app/bootstrap.php`.

22.1.1 What happens when somebody opens your website

It is helpful to imagine that every visit begins with a person handing a small note to the gallery's reception desk. The note is the web request: it may say that the visitor wants the home page, a particular gallery, one photograph, a thumbnail, a download, or an Admin action. The visitor does not need to know which file contains the answer, just as a visitor to a real archive does not need to know which cupboard contains a particular photograph. PHP Gallery receives the note through its public entrance, prepares the installation for this one visit, understands what was requested, and then passes the work to the part of the application that is best suited to answer it.

The first preparation step is called *bootstrap*. This does not create or change your photographs. It means that the application gathers the basic information needed to work safely: it reads the local configuration, connects the request to the correct database when database information is needed, prepares the language and security settings, and starts the visitor or administrator session when the requested action requires one. The application also checks general conditions that should apply before a page is produced, such as whether a feature is enabled, whether a request needs a security check, and whether a scheduled maintenance opportunity should be recorded. These checks happen before the requested gallery or control is allowed to continue, so the same protective rules do not need to be reinvented separately on every screen.

After preparation, PHP Gallery interprets the address. A visitor may use a clean address such as `/gallery/holidays/rome`, while another installation may use a compatible query-style address containing a page name. The application translates both forms into the same internal meaning. It recognizes that the first address asks for the public gallery called "holidays/rome" and that a request ending in a thumbnail filename asks for a smaller image copy rather than a normal page. This translation is called routing. Routing is not the act of opening a photograph; it is the act of deciding which kind of answer is needed and which information belongs to that answer.

Once the meaning is known, the application chooses a controller. A controller is the responsible reception clerk for one kind of request. A public-gallery controller prepares a gallery page, a media controller prepares an authorized image or original file, an Admin controller handles an administrative action, and a maintenance controller handles a repair or inspection task. The controller does not normally perform every detail itself. It asks specialized services to check access, find galleries and images, prepare thumbnails, load tags, validate a form, save a change, or create a download. Finally, the controller returns the appropriate result: an HTML page, a JSON response for an in-place Admin action, a media file, a download, or a redirect to another address.

This means that opening one gallery is a coordinated journey rather than a single file being

displayed directly. The request enters through one public door, the bootstrap prepares the application, routing identifies the requested destination, the controller coordinates the work, services apply the rules and gather the information, and the final response travels back to the browser. The browser then displays the result and may add interactive behavior such as the lightbox, search suggestions, voting, progressive thumbnail sharpening, or the Admin side panel. If JavaScript is unavailable, the server-rendered page and its normal links still provide the essential route wherever that feature supports a no-JavaScript path.

Important. For a gallery owner, this process explains why a direct image or thumbnail address is still protected. The address does not bypass the application simply because it ends in an image filename. It is routed through the same request preparation and access decision before the file is allowed to reach the visitor.

22.1.2 The three central steps: preparation, routing, and dispatch

The three central steps can be understood as preparation, routing, and dispatch. *Preparation* makes the application ready for the current visit. It reads the installation's instructions, establishes the context in which the request will run, and performs shared checks. *Routing* reads the requested address and turns it into a meaningful destination with its parameters, such as a gallery path, share token, page number, thumbnail size, or file format. *Dispatch* hands that prepared destination to the controller that knows how to complete it. The words are technical names for a simple sequence: get ready, understand the request, and give it to the right helper.

These steps are kept together near the beginning of the application because their order matters. Access and request-safety decisions must happen before private information is assembled. A thumbnail request must be understood as a thumbnail request before the application selects a generated file. An Admin form must be recognized as an Admin operation before its submitted values are accepted. Keeping the common preparation and hand-off steps in one central place helps the application behave consistently when the same photograph is reached from a gallery card, a lightbox, a direct link, a search result, or a download action.

22.1.3 Why one central entrance is useful

PHP Gallery uses one central public entrance for ordinary web requests. In technical language this is often called a *front controller*; for an owner, it is simply the main reception desk. It gives the application one reliable opportunity to prepare configuration, security, translation, sessions, and feature rules before any particular page is handled. The repository-root `index.php` is a compatibility entrance for hosting and local-development arrangements, while `public/index.php` is the normal public entrance. Both lead into the same bootstrap process, so choosing one supported web-root arrangement does not create a different application.

This arrangement also helps preserve the filesystem and database partnership described earlier. The address may identify a gallery path or image, but the application still resolves that identity through its catalog, checks the corresponding permissions, and then uses the actual gallery files when it is time to serve media. A request is therefore not trusted merely because its text resembles a folder name or a filename. The central entrance gives the application a consistent place to translate the request and apply the rules before it touches the requested result.

```
browser request
  -> index.php or public/index.php
  -> app/bootstrap.php
  -> cms_run()
```

```
   -> cms_route_from_request()
   -> controller function
   -> service functions
   -> HTML, JSON, media, archive, or redirect response
```

22.2 Controllers

Controllers are the reception desk of the application. They receive the request after the central entrance and routing steps have prepared it, look at what the visitor or administrator is asking for, and coordinate the answer. A controller does not represent a photograph or a gallery itself. It represents an action involving one of them: show this gallery, open this image, accept this upload, save this description, inspect this installation, or return the next part of an Admin panel.

For example, when a visitor opens a gallery, the controller first receives the selected gallery path and the visitor's request context. It helps establish which gallery the address refers to, asks the access rules whether the visitor may see it, and gathers the information needed for the page. When an administrator submits a form to rename a gallery, a different controller receives the submitted action, confirms that the administrator is signed in and that the request is genuine, and then asks the appropriate service to perform the change. The controller is therefore the coordinator of the journey, not the person who performs every specialist task.

Public gallery rendering resides principally in `app/controllers/public_gallery.php`. Other controller files receive uploads, media requests, maintenance actions, updates, downloads, and Admin forms. Dividing these entry points by the kind of request helps the application keep a public visitor's experience separate from private administration. It also means that a request for a thumbnail can receive a small, efficient media response instead of loading the full page machinery needed for a dashboard.

Controllers can return several kinds of answer. A normal page produces HTML for the browser to display. A side-panel action may produce a small HTML fragment or JSON information so the existing Admin screen can update in place. A media controller may send an authorized image file, while a download controller may prepare an archive. Some actions redirect the browser to a new address after a normal form submission. The result depends on the request, but the controller remains responsible for making sure that the right type of answer is returned.

The most important promise made by a controller is that it does not skip the rules simply because the request is direct or technically small. A request for a thumbnail, map point, original file, or background asset still passes through the relevant access decision. A controller also hands persistent changes to trusted services instead of duplicating file deletion, authorization, CSRF, or database logic in each individual form handler. For the owner, this means that using a normal gallery link, a direct photo link, or an Admin control leads through the same protective decisions.

22.3 Services

Services are the specialists behind the reception desk. If a controller is the person who receives a request, a service is the person who knows how to do one particular kind of work correctly. One service understands gallery access, another creates thumbnails, another reads and validates image metadata, another finds likely duplicates, and others handle uploads, updates, maintenance, downloads, search, translations, and telemetry. The service files live mainly under `app/services/`.

This separation is valuable because the same rule may be needed from several places. A photograph can be reached from a gallery card, a search result, a lightbox, a direct link, or a download operation.

Each of those entry points should not invent its own version of “is this visitor allowed to see the image?” Instead, the controllers ask the gallery-access service to make that decision. Likewise, thumbnail generation, thumbnail validation, and thumbnail cleanup are shared responsibilities rather than separate behaviors hidden inside upload, maintenance, and public rendering screens.

Consider a new photograph arriving through the browser. The upload controller receives the upload and passes it to upload-related services. Those services check the file, place it in the selected gallery, record the image in the catalog, read useful camera information, and arrange any requested thumbnail work. If the same photograph is later discovered because it was copied to the server by another method, discovery services can reuse the same image and metadata rules. The owner sees one consistent gallery even though the photograph entered through different doors.

Services also protect the difference between the original file and the information about it. The original remains in the gallery folders, while services coordinate the database catalog, generated thumbnails, checksums, descriptions, visibility, and maintenance state. When a gallery is deleted, the gallery-mutation service can coordinate the database and filesystem consequences. When a thumbnail is missing, the thumbnail-maintenance service can record or repair that condition without requiring every public page controller to understand the repair process.

Some services prepare information rather than change anything. A search service finds matching public content, a gallery lookup service resolves a public path, a rendering service prepares a media manifest, and a statistics service summarizes storage use. Other services perform deliberate changes such as saving a description or deleting an image. Keeping these responsibilities in named specialists makes it easier to tell whether a screen is merely reading the catalog or is about to change the collection.

For a gallery owner, the practical result is consistency. A setting changed in Admin is interpreted by the same focused service wherever it matters. An access choice affects pages and media routes together. A maintenance operation uses the same thumbnail and filesystem rules as an upload. The service layer is the application’s shared memory of how the gallery should behave.

22.4 Views and browser modules

Views are the part that turns prepared information into the page people see. They place the site name, navigation, gallery title, description, cards, controls, messages, and footer into an HTML response. A view should not decide whether a visitor is allowed to see a protected gallery or how a thumbnail is generated. Those decisions are made before the view receives its prepared information. In everyday terms, the view is the careful display cabinet: it arranges the material for the visitor without being responsible for deciding which material may leave the archive.

When a gallery page is prepared, a view may receive a selected gallery, its visible child galleries, permitted images, descriptions, tags, pagination links, thumbnail information, and the active theme. It can then present those elements consistently. The same layout helpers can be used for a home page, a public gallery, an Admin screen, or a side-panel fragment where appropriate, while each screen still has its own purpose and amount of detail.

Browser modules add behavior after the HTML reaches the visitor’s browser. They power the lightbox, search suggestions, voting, Admin side panels, upload progress, drag-and-drop actions, thumbnail upgrades, diagnostics, and other interactions. The server gives the browser the initial structure and the information needed to use it; the browser modules listen for actions and make the experience more immediate. For example, clicking a vote button can send a small request and update the displayed count without rebuilding the entire gallery page.

Browser code remains framework-free, which means PHP Gallery does not require a large client-

side application framework before a visitor can read a gallery. This keeps the public page understandable to ordinary browsers and helps the server-rendered HTML remain useful on its own. Public visitors receive a smaller set of browser features than signed-in administrators because an administrator's workspace needs controls for editing, progress, maintenance, and side-panel workflows that are not relevant to a normal visitor.

The Admin side panel deserves special mention. Its content can be replaced while the surrounding page remains open. The browser modules therefore use lifecycle-aware setup and teardown behavior: a module can attach to a newly inserted form, stop listening to an old fragment, and attach again to the refreshed copy. Event delegation allows a stable parent area to recognize actions from controls that were added later, so the administrator can save an item, keep the panel open, and continue working without a full-page navigation.

The division between views and browser modules also provides a safety net. The server can render links, forms, titles, alternative text, and essential image choices before JavaScript runs. JavaScript then improves speed and convenience where the browser supports it. This is why a visitor should still be able to open a gallery, follow a direct link, read its content, and navigate its important parts even when scripts are blocked, delayed, or unavailable.

22.5 Public selected-gallery render pipeline

22.5.1 Server render and browser enhancement

When a visitor opens a gallery, PHP Gallery follows a deliberate render pipeline. First it identifies the selected gallery and resolves the effective access rules, including visibility, parent-gallery protection, password state, share access, and any age-related restriction. The application performs this work before it constructs public media addresses. A visitor therefore does not receive a useful-looking page containing links to material that the visitor is not allowed to open.

The application then prepares the gallery's shape. It counts or estimates the information needed for navigation, determines the current page of results, loads the direct child galleries, and selects the image rows that are visible in the current scope. It also prepares titles, descriptions, dates, tags, votes, ordering, and relevant display choices. A large collection is not treated as one enormous undifferentiated list: pagination allows the visitor to move through it in manageable sections.

Next, the application prepares the media that the visible cards will need. It gathers thumbnail metadata in batches rather than asking the database the same small question once for every card. It creates request-local media manifest entries describing the candidates that are actually available and permitted for each image. This preparation allows the page to expose suitable image choices without repeatedly probing every possible file while the visitor is waiting for the first screen.

The selected-gallery controller then gives the prepared information to the presentation layer. The response includes the page title and other search-engine information, branding, navigation, breadcrumbs, child-gallery cards, photograph cards, pagination, lightbox configuration, and the footer assets. Diagnostics and privacy-controlled telemetry may also receive carefully selected information about the render. These pieces are assembled for this request; they do not mean that the application has permanently copied the original photographs into the page or database.

The browser receives a useful server-rendered starting point. It can read the title and descriptions, discover the initial image candidates, follow links, and preserve the page's semantic structure before JavaScript has finished loading. Browser modules then enhance that starting point: search can respond as the visitor types, votes can update in place, the lightbox can open without a new page, and the progressive renderer can request larger candidates when appropriate. The server remains responsible for access and for producing safe initial markup; the browser supplies

convenience and carefully bounded enhancement.

Server-side rendering gives browsers early semantic content and image discovery. JavaScript enhances search, votes, lightbox behavior, thumbnail policy, diagnostics, and authenticated controls. This division supports accessibility, crawlers, direct links, and graceful operation on constrained clients.

22.5.2 A visitor's example journey

Imagine that Anna opens a shared link to *Travel / Rome*. The request first enters the central application entrance. Bootstrap prepares the installation and the request context. Routing recognizes the gallery path and passes it to the public-gallery controller. The controller asks the access service whether Anna may see the gallery, resolves the gallery record, and determines which child galleries and photographs belong on the requested page.

The services then prepare the details that make the page useful. Gallery and image information provides the names and descriptions, pagination chooses the visible part of the collection, tag and vote services provide related controls, and thumbnail services identify the available display copies. The view arranges those results into the page. When the response reaches Anna's browser, the first cards can begin to appear using server-provided image information. If JavaScript is available, the lightbox and other enhancements become active; if it is not, the ordinary links and server-rendered content still provide the essential journey.

If Anna clicks a photograph, the new request or browser action does not abandon the original protections. The selected image is resolved again, access is checked again where required, and the appropriate preview or original is chosen for that purpose. If she later opens the same link from a different browser, the application makes a fresh decision based on that visitor's access state rather than trusting that the address was previously used successfully. This repeated discipline is what allows friendly, shareable addresses to coexist with private and protected galleries.

23 Optional background: what the database remembers

The database is the gallery's catalog. It remembers facts and choices that would be awkward to store in image filenames: descriptions, privacy, order, tags, votes, dates, public addresses, accounts, and maintenance history. This section offers an overview for owners preparing backups or discussing the installation with a hosting provider. Detailed column definitions remain in the separate database reference [3].

23.1 Principal domains

Domain	Persistent responsibility
Users and authentication	Administrator identity, password hashes, recovery email, external identity links, reset tokens, and scoped credentials.
Galleries	Folder path, hierarchy, public path, metadata, visibility, access, branding, presentation overrides, and operational ordering.
Images	Gallery ownership, relative path, filename, metadata, dimensions, visibility, checksum, EXIF data, ordering, and derived state.
Tags	Reusable tag identity plus gallery and image relationships.
Votes	Gallery and image scoring records with viewer association.
Thumbnails	Valid generated variant inventory, dimensions, format, status, and derivative version.
Settings	Scalar application, theme, feature, update, maintenance, and rendering values.
Audit and telemetry	Administrator events, operational diagnostics, privacy-controlled measurements, and rollups.
Duplicate ledger	Per-administrator canonical pair decisions and exact-gallery suppression rules.
Jobs and tokens	Bounded browser, detector, migration, WebDAV, reset, sharing, and maintenance state where persistence is required.

23.2 Relations and cascades

Foreign keys express ownership where schema evolution and compatibility permit it. Cascades remove dependent workflow records when their parent identity disappears. Content deletion continues through application services so filesystem and database effects remain coordinated. Unique constraints encode identities such as canonical duplicate pairs, exact administrator-gallery rules, slugs, tokens, and metadata variants.

23.3 Application settings

The `app_settings` table stores key-value configuration. `app_setting()` reads values, `set_app_setting()` persists normalized values, and focused services interpret domain-specific settings. Public language configuration uses `public_language` for the site default, `public_language_selector_enabled` for the default-on viewer override feature, and `public_language_selector_languages` for its ordered non-empty language list. The public thumbnail renderer uses `public_thumbnail_rendering_mode` with `responsive` and `progressive` values.

24 Optional reference for developers

This section supports people changing the PHP Gallery software itself. Gallery owners can continue directly to Section 25 for publication and maintenance checks.

24.1 Repository organization

Location	Responsibility
`app/controllers/`	HTTP controllers and response coordination.
`app/services/`	Reusable domain and infrastructure services.
`app/views/`	Layout and reusable server-rendered view helpers.
`app/lang/`	Server and browser translation catalogs.
`database/migrations/`	Timestamped schema evolution.
`public/assets/`	Framework-free JavaScript, CSS, and public assets.
`scripts/`	Migration, integrity, deployment, and maintenance utilities.
`tests/`	Standalone PHP and focused JavaScript tests.
`winapp/`	Windows-specific tools and integrations.

24.2 Coding conventions

New PHP files use strict types and four-space indentation. Controllers coordinate request behavior, while services own reusable logic. New JavaScript and CSS remain framework-free and belong under public assets. Migrations use timestamp-prefixed filenames. Existing explanatory comments, docstrings, and PHPDoc receive preservation and correction as behavior evolves.

Atomic modules should own one cohesive responsibility. Shared contracts deserve normalized inputs, explicit outputs, and focused tests. Dynamic Admin and public fragments require lifecycle-safe setup and teardown.

24.3 Adding a feature

A typical feature change follows this sequence:

1. Trace current implementation and identify the owning controller, services, views, browser modules, persistence, and tests.
2. Define access, CSRF, scope, fallback, migration, and no-JavaScript contracts.
3. Add focused services and route integration.
4. Add append-only migration work where persistent schema changes are required.
5. Add translated interface strings.
6. Add focused automated tests and documented manual verification.
7. Update architecture, code map, database, testing, and user guidance according to their ownership.

8. Regenerate integrity metadata after final runtime changes.
9. Review the complete diff and run the relevant local test suite.

25 Checking your gallery before publication

Testing for a gallery owner means walking through the site as the intended audience. A technically healthy page can still contain a private location, an unclear title, an incorrect date, an accidental download permission, or an awkward mobile layout. A short human review catches these presentation and privacy concerns.

25.1 A visitor-centered publication check

Open a private browser window and follow the same link you plan to share. Check:

1. The title and first paragraph explain the collection.
2. The chosen cover remains clear at a small size.
3. Child galleries appear in a sensible order.
4. Photograph titles and descriptions match the visible subjects.
5. Private photographs and galleries stay hidden.
6. Password or share access works from a fresh session.
7. GPS maps reveal only intended locations.
8. The first page loads comfortably on a phone.
9. The large viewer, forward and backward controls, and close action feel natural.
10. Search, tags, voting, and downloads behave according to the gallery's purpose.

Ask another person to try the gallery without instructions. Their first click and first question often reveal navigation or wording that felt obvious only to the owner.

25.2 Automated checks for software maintainers

Developers can run the project's executable test scripts after changing application code. The aggregate runner covers the project suite, while focused scripts verify particular features.

```
php tests/run.php
php tests/public_thumbnail_rendering_model_test.php
php tests/public_thumbnail_markup_test.php
php tests/duplicate_photo_detector_test.php
php tests/duplicate_photo_ledger_test.php
php tests/migration_consistency_test.php
node tests/progressive_thumbnail_renderer_test.mjs
```

25.3 Manual browser verification

25.3.1 Network and interaction checks

Browser-dependent software checks use representative data, anonymous and authenticated sessions, an empty cache, mobile and desktop viewports, JavaScript-disabled operation where relevant, reduced-motion preferences, and network throttling.

For public thumbnail rendering, inspect the Elements and Network panels. Responsive mode should expose its complete active candidate set. Progressive mode should expose a small active candidate and inert larger candidates before activation, then perform at most the documented bounded upgrades for relevant cards. Check layout stability, cache reuse, error fallback, lightbox interaction, maps, voting, protected media, and direct links.

25.4 Release hygiene

A release review confirms runtime version consistency, ordered migration compatibility, manifest hashes, documentation, translations, browser cache-busting, tests, package exclusions, and user-data safety. The maintainer starts from a clean release branch, compares it with the exact previous release tag, reviews every intervening commit and path, and records anything intentionally excluded. The runtime version in `app/bootstrap.php`, the `release-metadata.json` key and tag, newest `PATCH_NOTES.md` heading, documentation markers, intended archive name, and final Git tag must all agree.

New migrations are reviewed in timestamp order and exercised through migration-consistency and legacy-runner tests. The maintainer rebuilds this PDF with the complete `docs/LATEX_BUILD.md` sequence and inspects its title page, release overview, contents, bookmarks, index, and changed feature sections. Every changed PHP file receives `php-l`; changed JavaScript and Node fixtures receive syntax checks; focused feature tests, translation consistency, documentation coverage, the complete PHP suite, and `gitdiff--check` must pass.

Only after the final source and documentation edit does the maintainer run `phpscripts/generate_manifest.php` and its `--check` mode. The deployment helper then creates the requested local folder or ZIP; it only packages files and does not execute PHP or repair a stale manifest. Inspect the archive listing and confirm it contains the current PDF, patch notes, release metadata, migrations, and `app/core-manifest.json`, while protected configuration, caches, logs, temporary files, local tools, deploy output, and user media follow the packaging policy. Recheck the manifest after packaging, create the release commit and annotated `v_<version>` tag from the verified tree, and smoke-test an updater upgrade from the previous stable release. Missing local dependencies must be reported with exact blocked commands and environment details rather than calling the release fully verified.

26 Making the gallery feel fast

Visitors experience speed as a sequence of impressions: the page begins to appear, the first photographs become recognizable, scrolling responds, controls work, and larger photographs open without an uncomfortable pause. Total loading can continue quietly after the page already feels useful.

The largest practical influences are the visitor's connection, distance to the server, number of photographs shown at once, thumbnail sizes, large branding images, and the selected thumbnail rendering mode. A gallery owner can improve the experience without reading technical timing reports.

26.1 Practical ways to improve speed

- Keep pagination enabled for very large galleries.
- Generate the configured thumbnails before sharing a new large collection.
- Use an appropriately sized background and branding image.
- Choose covers that remain effective as compact previews.
- Test responsive and progressive thumbnail modes with the actual audience and gallery layout.
- Use a hosting location reasonably close to most visitors.
- Enable normal web-server compression and caching with help from the hosting provider.
- Review the site as an anonymous visitor because signed-in Admin pages load additional controls.

26.2 A simple slow-connection test

Open the gallery in a browser's private window, select a mobile screen size, enable a throttled network profile in developer tools, and reload with an empty cache. Observe when the title appears, when the first row becomes recognizable, when scrolling feels ready, and when the first large photograph opens. Repeat with the alternative thumbnail mode and compare the experience.

26.3 Technical measurements for maintainers

Developers and hosting administrators can use public render profiling and browser diagnostics for precise comparisons. Useful measurements include:

Useful measurements include:

- time to first byte and server render spans;
- first contentful paint and largest contentful paint;
- time until the first visible cards contain images;
- transferred bytes before interaction;
- number and priority of image requests;

- layout shift;
- responsive candidate selected by the browser;
- progressive small-to-large transfer sequence;
- lightbox activation and first-preview latency;
- bytes transferred after the initial page becomes usable.

The development profiler and captured benchmark records support comparisons across renderer modes and gallery layouts. Representative testing includes high-density screens and constrained connections because those screens can request larger thumbnail candidates.

27 Troubleshooting

27.1 Application does not start

Confirm the PHP version, required extensions, configuration location, database connectivity, writable paths, and web-root routing. Review PHP and web-server logs. Run syntax checks on recently changed PHP files and inspect pending migrations.

27.2 Gallery or images are missing

Confirm filesystem paths, gallery discovery state, database records, visibility, parent access rules, password unlock state, NSFW policy, supported file type, and image relative path. Test as both administrator and anonymous visitor.

27.3 Thumbnails are missing or soft

Review thumbnail metadata and maintenance reports. Confirm generated sizes, format support, configured bounds, source geometry, derivative status, and public endpoint authorization. In responsive mode, inspect the browser-selected candidate and `sizes`. In progressive mode, inspect the small candidate, queue state, selected replacement, and decode result.

27.4 Admin actions leave the side panel

Confirm the matching JavaScript module loaded, delegated handlers recognized the form, AJAX headers and response contracts are correct, the returned fragment contains expected lifecycle markers, and generic Admin form handlers exclude feature-owned forms. Direct POST behavior can still provide the documented fallback route.

27.5 Migration fails

Record the exact migration filename, database error, server version, existing schema state, and migration audit rows. Use available dry-run or inspection tools. Preserve the database before repair and follow the migration's idempotent compatibility path.

27.6 PDF manual compilation fails

Run the commands in `docs/LATEX_BUILD.md`. Confirm `pdflatex` and `makeindex` are installed. A first MiKTeX run may request common packages. Delete ignored intermediate build files after a failed package transition and repeat the documented sequence.

28 Backup and recovery checklist

1. Export the complete application database.
2. Copy `galleries/` with source files and gallery-owned assets.
3. Copy local configuration and relevant installation secrets securely.
4. Preserve custom theme assets and custom CSS.
5. Record the application version and pending migration state.
6. Store backups outside the active web tree.
7. Test restoration periodically in an isolated environment.
8. Verify public routes, protected access, Admin login, thumbnails, uploads, and migrations after restoration.

Updater-created backups preserve overwritten application files for update recovery. A complete operational backup also includes the database, gallery tree, local configuration, and installation-owned assets.

29 Glossary

AJAX Browser request used to update a page region while preserving the surrounding interface.

Branch scope A gallery and its descendants, when the selected feature defines recursive scope.

Canonical pair Two image identifiers stored in a stable low-to-high order.

CSRF token Session-bound value used to validate intentional state-changing form submissions.

Derivative Generated media representation such as a JPEG or WebP thumbnail.

EXIF Camera and capture metadata embedded in supported image files.

Integrity manifest Versioned list of core paths and expected hashes.

Lightbox Fullscreen or overlay photo viewer with navigation and related controls.

Media manifest Request-local rendering data prepared from thumbnail metadata for visible images.

NSFW guard Age and gallery/image policy controlling access to restricted media.

Progressive sharpening Small-first thumbnail pipeline that activates a decoded larger candidate later.

Responsive selection Browser-native candidate selection from an immediately active source set.

Selected gallery Gallery represented by the current public route.

Side panel Admin right-side contextual interface loaded and refreshed through fragment requests.

Thumbnail bounds Gallery-aware minimum and maximum generated sizes eligible for selection.

30 References

References

[1] PHP Gallery project, *README.md*, local product overview, feature guide, requirements, and installation documentation, version 0.90.

[2] PHP Gallery project, *ARCHITECTURE.md*, local application architecture and subsystem lifecycle reference, version 0.90.

[3] PHP Gallery project, *DATABASE.md*, local migration and persistent schema reference, version 0.90.

[4] PHP Gallery project, *CODEMAP.md*, local source ownership and maintenance map, version 0.90.

[5] PHP Gallery project, *docs/ADMIN_SETTINGS_INVENTORY.md*, canonical global Settings ownership, fallback, sensitivity, and migration inventory, version 0.90.

[6] PHP Gallery project, *TESTING.md*, local automated and manual verification guide, version 0.90.

[7] PHP Gallery project, *AGENTS.md*, local repository contribution and security guidelines.

[8] The PHP Documentation Group, *PHP Manual*, https://www.php.net/manual/en/.

[9] Oracle Corporation, *MySQL Reference Manual*, https://dev.mysql.com/doc/.

[10] MariaDB Foundation, *MariaDB Server Documentation*, https://mariadb.com/docs/server/.

[11] WHATWG, *HTML Living Standard: Images*, https://html.spec.whatwg.org/multipage/images.html.

[12] OWASP Foundation, *Web Application Security Guidance*, https://owasp.org/.

Index